



Bachelorarbeit

*Auf dem Weg zum selbstfahrenden Fahrrad: Analyse und
Implementierung einer Regelung der Balance
(und der Geschwindigkeit)*

vorgelegt von:	Amine Karabila
Matrikelnr.:	6611011
Telefon:	0157 34936593
Gutachter:	Prof. Dr.-Ing. Lars Hedrich
Zweitgutachten:	Prof. Dr. Uwe Brinkschulte
Betreuer:	Prof. Dr.-Ing. Lars Hedrich

Abgabetermin:	15.08.2022
----------------------	------------

Bitte dieses Formular zusammen mit der Abschlussarbeit abgeben!

Erklärung zur Abschlussarbeit

**gemäß § 25, Abs. 11 der Ordnung für den Bachelorstudiengang Informatik
vom 06. Dezember 2010:**

Hiermit erkläre ich

Karabila Amine
(Nachname, Vorname)

Die vorliegende Arbeit habe ich selbstständig und ohne Benutzung
anderer als der angegebenen Quellen und Hilfsmittel verfasst.

Frankfurt am Main, den



Unterschrift der/des Studierenden

Zusammenfassung

Mit dieser Arbeit wurde eine Regelung zur Selbstbalancierung eines Fahrrads entwickelt. Dabei wird die Selbstbalancierung durch drei P-Regler realisiert, diese wird durch einen Lagesensor (IMU), einem Encoder (MD49) und einem 48V Hinterradantrieb beeinflusst. Die dabei erhaltenden Parameter werden dann für die Realisierung eingesetzt.

Beim Start dieser Regelung kommt es zur kontinuierlichen Abfrage der IMU und einer Weitergabe der Winkelwerte an den Beaglebone Black (BBB). Dieser übergibt dem Encoder im Anschluss den anzufahrenden Encoder Wert, der durch die Pulsweitenmodulations (PWM) die Motorsteuerung des Lenkers wie gewünscht gegensteuert. Dabei werden Begebenheiten der Fahrdynamik ausgenutzt, sodass eine vereinfachte Regelung erstellt werden konnte.

Erkenntnisse werden dabei aus festgehaltenen Rohdaten ermittelt und Rückschlüsse auf eingesetzte Regeler und Parameter erstellt.

Inhaltsverzeichnis

ZUSAMMENFASSUNG	IV
1 EINFÜHRUNG	8
1.1 MOTIVATION	9
1.2 AKTUELLE PROBLEMSTELLUNG	9
1.3 AUFBAU DER ARBEIT	10
2 AKTUELLER FORSCHUNGSSTAND	11
2.1 DEFINITION DES AUTONOMEN FAHRENS	11
2.2 STAND DER FORSCHUNG	12
2.3 MÖGLICHE EINSATZGEBIETE	13
3 AUFBAU, ERWEITERUNGEN UND GESAMTKONZEPT	14
3.1 MODUL UND SCHALTPLAN ÜBERSICHT	14
3.2 KOMPONENTEN	16
3.3 BEAGLEBONE BLACK	21
3.4 PULSDAUERMODULATIONS VERFAHREN	22
3.5 PROGRAMMABLE REAL-TIME UNIT	24
3.6 ANPASSUNG DER STÜTZRÄDER	26
4 EXPERIMENTE	28
4.1 VERSUCHSAUFBAU	29
4.2 OPTIMIERUNG	30
4.3 GEGENÜBERSTELLUNG BEIDER REGELUNGEN	32
5 EVALUATION	39
5.1 BEWERTUNGSKRITERIEN	39
5.2 MESSDATEN	40
5.3 BEWERTUNG DES ERSTEN REGLERS	41
5.4 BEWERTUNG DES ZWEITEN REGLERS	42
6 FAZIT	49
6.1 ERGEBNISSE	49
6.2 AUSBLICK	49
LITERATURVERZEICHNIS	A
ABBILDUNGSVERZEICHNIS	B
ABKÜRZUNGSVERZEICHNIS	C

1 Einführung

In Zukunft soll es möglich sein, sich während dem Autofahren auf weitere Aktivitäten zu fokussieren. Man könnte mitten im Verkehr E-Mails beantworten oder sich an einem Online-Meeting teilnehmen. Diese Möglichkeiten sind im Alltag der Menschen noch nicht angekommen, wobei sich diese Technologie rasant weiterentwickelt.

Zur selben Zeit gibt es eine Menge anderer Fortbewegungsmittel, bei denen sich das Prinzip des autonomen Fahrens, noch in den Kinderschuhen¹ befindet. Eines dieser Fortbewegungsmittel stellt das Fahrrad dar. Rund 80% [1] der Deutschen besitzen ein Fahrrad, in ärmeren Ländern, wie z.B. in Namibia und Sambia, ist es jedoch ein Fortbewegungsmittel, welches am meisten genutzt wird [2]. Daher ist ein stetiges Vorantreiben und Weiterentwickeln solcher und ähnlicher Arbeiten, ein wichtiger Bestandteil für die Erstellung von nachhaltigen Konzepten. Da Fahrräder durch ihr eigenes deutlich leichteres, Eigengewicht (im Gegensatz zu PKWs) keinen hohen Verbrauch an benötigten fossilen Rohstoffen vorzuweisen haben, wären diese eine gute Option für das Bewältigen von langen Strecken. So wären sie eine gute Alternative zu Autos und Motorräder, die einen hohen Verbrauch und damit einhergehende Kosten haben. Da es sich bei diesen Rohstoffen meist um Benzin oder Diesel handelt und daher um einen begrenzten Rohstoff, sind Alternativen wie z.B. Elektroantriebe eine gute Wahl für ein solches Konzept. Durch das Ausbauen von erneuerbaren Energien in Afrika durch die hohen Temperaturen, ist eine unverbrauchbare Energiequelle vorhanden. Als Beispiel sei hierbei das weltweit größte Solarkomplex [3] in Marokko zu erwähnen, das mit einer Fläche von 650 Hektar, 400 Gigawattstunden Strom pro Jahr erzeugen kann. Dabei handelt es sich um eine Strommenge, die durchschnittlich 400.000 Menschen im Jahr in Marokko verbrauchen. Dadurch versucht das Land sich von Fossilien Energien zu befreien.

¹ Redewendung dafür, dass es sich um ein nicht ausgereiftes Stadium einer Entwicklung handelt.

1.1 Motivation

Durch das vom EU-Ministerrat vorgeschlagene Verbrenner Verbot ab 2035, können Autos mit einem Verbrennungsmotor ab diesem Zeitpunkt nicht mehr verkauft werden [4]. Aus diesem Grund müssen die Automobilhersteller vollständig auf alternative Kraftstoffe setzen. Dadurch sind immer mehr Menschen dazu geneigt, mit dem Fahrrad zur Arbeit zu fahren. Hierbei stellt die Niederlande ein ideales Vorzeigbeispiel dar. Denn die Infrastruktur der Niederlande [5] ist auf das Fahrradfahren ausgerichtet, hier bietet das Fahrradfahren eine gute Möglichkeit, um auf das Auto zu verzichten. Viele Straßen haben eigene Straßenübergänge und Ampeln für Fahrradfahrer eingerichtet, mithilfe dessen man an Orte gelangt, die mit dem Auto nicht zu erreichen sind.

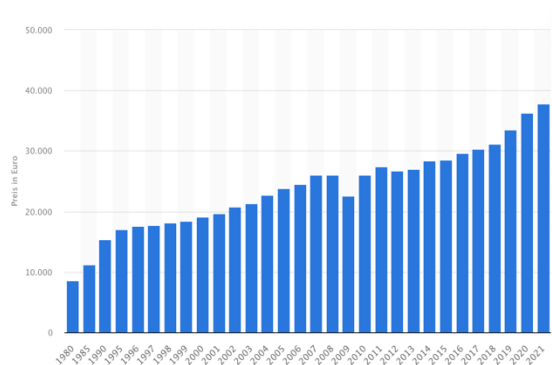


Abbildung 1: Preisentwicklung von Fahrrädern (2017 - 2021)

<https://de.statista.com/statistik/daten/studie/36408/umfrage/durchschnittliche-neuwagenpreise-in-deutschland/>

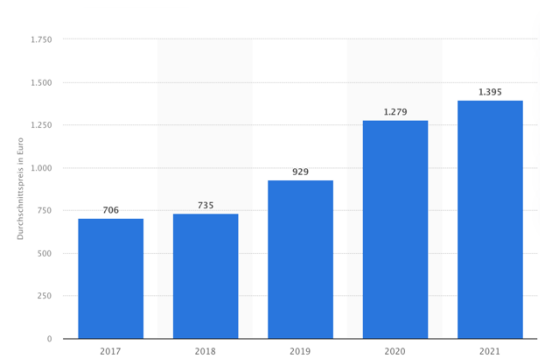


Abbildung 2: Preisentwicklung von Autos (1980 bis 2021)

<https://de.statista.com/statistik/daten/studie/1107744/umfrage/durchschnittspreise-verkaufter-fahrraeder-in-deutschland/>

Ein Neuwagen in Deutschland kostet im Schnitt 37.790€ - Tendenz steigend. Im Gegensatz dazu kostet ein Fahrrad im Jahr 2021 im Schnitt 1.395€. Trotz der ungleichen Zeitintervalle beider Diagramme, sind in beiden Fällen steigende Kaufpreise zu erkennen. Viele Menschen in Städte sehen es daher nicht mehr ein, sich mit dem Auto durch eine gut vernetzte Kleinstadt zu bewegen und ziehen es daher vor, auf das Fahrrad als Fortbewegungsmittel auszuweichen. Eine solche Tendenz haben auch viele Großkonzerne erkannt und streben Forschungen in diesen Bereichen an. Ein Vorreiter für die Entwicklung erster selbstfahrender autonomer Fahrräder ist Google. Das Technologieunternehmen hat mit einem scherzhaften Video [6] ein Fahrrad präsentiert, das sich völlig autonom im Straßenverkehr eingliedern ließ, wodurch sich viele Forschende dazu antreiben lassen haben, sich mit dieser Thematik ernsthaft zu beschäftigen.

1.2 Aktuelle Problemstellung

Ziel dieser Arbeit ist es, mittels der vorherigen Bachelorarbeit [7] als Fundament, eine Regelung für ein selbstfahrendes Fahrrad zu entwickeln. Das Fahrrad soll sich dabei, durch erstes Mitführen, komplett selbst balancieren. Da sich das Fahrrad zu Beginn dieser Arbeit, nicht von selbst antreiben ließ, musste der Hinterradantrieb mit dem BBB verschaltet werden. Durch das Verschalten mit dem BBB, steuerte dieser den Hinterradantrieb und somit wurde das Fahrrad nicht mehr nur ausschließlich mit dem Gasgriff gesteuert. Ein weiterer Schritt für den Hinterradantrieb, war das Balancieren im Stand. Dabei sollte das Fahrrad durch ein stetiges Hin und Herfahren, die Möglichkeit erlangen die Balance zu halten. Leider konnte diese Idee nicht umgesetzt werden.

Zu Beginn dieser Arbeit, konnten durch „Demo Codes“ der vorherige Arbeit (siehe Code/Amine_Karabila/Democodes), die Funktionalität der Komponenten getestet und deren Funktionalität verstanden werden. Diese Democodes konnten den Lenkmotor antreiben und den Encoder des MD49 auslesen. Das Auslesen erwies sich im Laufe dieser Arbeit als Grundbestandteil, für die Funktionalität der Regelung und als wichtiger Parameter.

Ein weiteres Ziel war das Erlangen einer Übersicht über die Schaltung des Fahrrades, da diese zu Beginn sehr unübersichtlich war. Nach ersten Zeichnungen der Schaltung, wurde diese dann im Anschluss im LTSpice² Programm übertragen und dient jetzt nun als optimale Übersicht des Systems.

1.3 Aufbau der Arbeit

Mit dieser Arbeit wird dem Leser zu Beginn der Begriff des „autonomen Fahrens“ nähergebracht und im Anschluss daran wird der heutige Stand der Forschung thematisiert. Dabei werden abgeschlossene Arbeiten vorgestellt, welche eine ähnliche Zielsetzung beinhalten und erklärt, inwiefern Schnittpunkte bestehen. Dabei werden die verschiedenen Herangehensweisen dieser Arbeiten besprochen, gleichzeitig wird dem Leser die Vielfalt und Anzahl an verschiedenen Möglichkeiten ein solches komplexes System zu planen und zu erstellen, nähergebracht. Darauffolgend werden verschiedene Einsatzmöglichkeiten eines selbstfahrenden Fahrrads erklärt und die damit einhergehenden Vereinfachungen verschiedener alltäglicher Probleme sowie auch auf unkonventionellen Einsatz-gebieten nähergebracht.

Im dritten Kapitel wird das Fahrrad, sowie die verbauten Komponenten behandelt. Die verschiedenen Komponenten, sowie die Kommunikationsprotokolle werden dabei erklärt und anhand von Einsätzen nähergebracht. Hierbei wird versucht, dem Leser zu vermitteln, wie die einzelnen Komponenten funktionieren und wie sie für den benötigten Zweck einzuordnen sind. Abschließend werden die Stützräder behandelt, hierbei wird versucht, den einzustellenden Winkel herzuleiten. Dabei fungieren die verbauten Stützräder als eine Art Sicherung der Komponenten.

Im vierten Kapitel werden die verschiedenen Testfahrten behandelt. Vom Versuchsaufbau hinweg über die Optimierung, werden die Regelungen bis ins kleinste Detail beschrieben und gegenübergestellt, um sie für eine Evaluation vorzubereiten. Es wird erklärt, wie man zu den jeweiligen Entscheidungen für bestimmte Vorgehensweisen in den Testfahrten kam und inwiefern Optimierungen der Laufzeit, eine Verbesserung des Gesamtsystem eingebracht haben. Dabei wurden eingesetzte Parameter erklärt und entsprechende Bewertungskriterien gesetzt.

Im fünften Kapitel kommt es dann zu der Auswertung der jeweiligen Testfahrten und einer Bewertung in der Tauglichkeit für eine Weiterentwicklung mit den genutzten Regelungen. Eine Übersicht der Grafiken der Testfahrten mit dem jeweiligen Parameter werden aufgezeigt und bewertet. Es wird versucht dem Leser eine Verbindung zwischen der Fahrdynamik, sowie eine Wahl eines Reglers zu erklären und zu rechtfertigen. Im sechsten Kapitel wird dann ein Fazit aus der Arbeit gezogen und ein Ausblick für kommende Arbeiten gestellt. Hierzu werden viele verschiedene Konzepte aufgezeigt, welche eine stabilere und erweiterte Fahrt mit dem Fahrrad ermöglichen.

² Eine kostenlose Software zur Erstellung von Schaltungssimulationen.

2 Aktueller Forschungsstand

2.1 Definition des autonomen Fahrens

Aktuell werden an vielen Universitäten Forschungen darüber angestellt, die Automobilbranche noch autonomer zu gestalten. Dabei entsteht eine Unterteilung in fünf Level, welche die Komplexität des autonomen Systems widerspiegeln [7]:

- Level 1: Assistiert,
- Level 2: Teilautomatisiert,
- Level 3: Hochautomatisiert,
- Level 4: Vollautomatisiert,
- Level 5: Autonom:

Beim assistierten Fahren wird z.B. der Tempomat gerne als Beispiel genommen, der die vom Fahrer eingestellte Geschwindigkeit konstant hält. Schon im Jahr 1962 wurde diese Technologie das erste Mal in einem Mercedes-Benz 450 SEL verbaut [8]. Beim teilautomatisierten Fahren, kann teilweise der Fahrer vom Bewegen des Fahrzeugs abgelöst werden, wie z.B. das Halten der Spur, das Bremsen, sowie das Beschleunigen des Fahrzeugs. Dabei muss der Fahrer den Blick stets auf den Verkehr richten und darf die Kontrolle über das Fahrzeug nicht komplett abgeben.

Beim hochautomatisierten Fahren allerdings, ist ein kontrollierender Blick des Fahrers nicht mehr nötig. Trotzdem ist dieses Unterfangen zeitlich limitiert, da sich hierbei das autonome Fahren für längere Strecken, noch nicht bewährt hat.

Das vollautomatisierte Fahren ist ein Level, auf dem der Fahrer die Kontrolle komplett abgeben kann. Dadurch ist es dem Fahrer möglich sich völlig auf andere Aktivitäten zu konzentrieren. Der Fahrer kann nun schlafen, das Smartphone nutzen oder arbeiten. Dabei erkennt das System seine Grenzen³ und bringt sich frühzeitig in einen sicheren Zustand. Ein solcher sicherer Zustand kann beispielsweise das rechte Heranfahren an den Seitenrand sein, um sich vom Verkehr zu entfernen.

Das autonome Fahren benötigt einen solchen sicheren Zustand nicht, da es alle Verkehrssituationen bewältigen kann. Dabei ist das Fahren mit oder auch ohne Passagiere möglich.

Zurzeit befinden sich Verkehrszugelassene Fahrzeuge im Level 2 des assistierten Fahrens. Als Beispiel ist hier das Tesla Model 3 zu nennen, das seit Jahren eine „Full Self Driving“ (FSD) Option anbietet, wobei der Fahrer zu jeder Zeit eingreifen können muss [9]. Er kann dabei seinen anzufahrenden Zielort eingeben, worauf das Fahrzeug sich teilautomatisiert, dort hinbewegen kann.

Eine solche Einteilung der Level gibt es bis zum heutigen Tage weder für Fahrräder noch für ähnliche Fahrzeuge. Es ist jedoch möglich sich aber an die für Autos orientierte Einteilungen zu halten.

³ Als Grenze sind dabei Situationen gemeint, die der Künstlichen Intelligenz (KI) des Autos nicht bekannt sind.

2.2 Stand der Forschung

Der aktuelle Stand der Forschung für das automatisierte Fahren von Fahrrädern, wurde in den bisherigen Recherchen nur vereinzelt gefunden. Viele Forscher sehen vermutlich keinen Mehrwert darin, sich mit einer Balancierfähigkeit eines Fahrrades auseinandersetzen zu müssen, wenn man gleichzeitig sich mit dreirädrigen Lastenräder⁴ oder ähnlichen Fahrzeugen beschäftigen kann. Daher wird die Beschäftigung mit dem Aspekt der Balance als irrelevant angesehen. Für Fahrräder mit drei Auflagepunkten, sind einige Forschungsarbeiten zu finden. Eine dieser Arbeiten ist die Arbeit der Universität Magdeburg [10]. In dieser Arbeit wurde die Fähigkeit, sich im Verkehr gut eingliedern zu lassen, fokussiert. Die Idee war es, ein selbstfahrendes Lastenrad zu entwickeln, welches die Möglichkeit besaß, Waren kostengünstig von A nach B zu transportieren. Dabei war es eine wichtige Bedingung für die Forscher, dass es die Möglichkeit besitzt, Unfälle vorherzusehen und diese zu vermeiden. Bei der Zustellung des Fahrrades wurde sich an der Sensorik von autonomen Autos bedient, da sich diese Sensorik für die Orientierung nicht sehr stark unterscheidet. Die Forscher haben in einem Artikel [11] dabei folgendes gesagt: „Das autonome Bike muss sich anhand von Laserscannern, Kameras und Sensoren orientieren, permanent die eigene Position berücksichtigen, nach Hindernissen Ausschau halten und anderen Verkehrsteilnehmern ausweichen. Das ist eine Herausforderung“, sagt Stephan Schmidt, Beteiligter dieser Forschungsgruppe. Gleichzeitig arbeitet das Massachusetts Institute of Technology (MIT) an einer ähnlichen Forschung wie der Lieferdienst Uber. Jedoch sind alle drei Ansätze von einer Serienreifen noch weit entfernt.

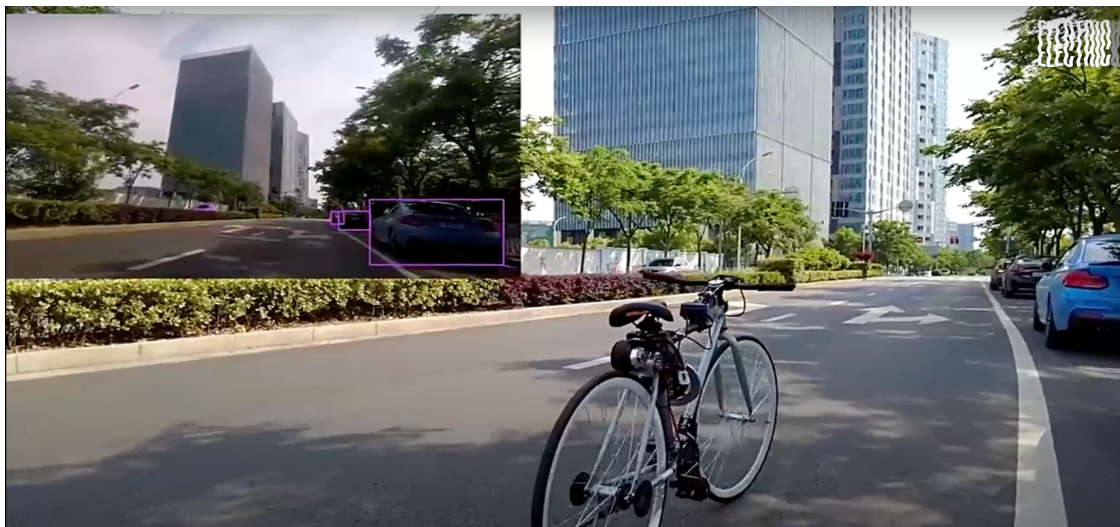


Abbildung 3: Aufnahme einer Testfahrt des Autonomen Fahrrades (Screenshot)

Quelle: <https://www.youtube.com/watch?v=kCL2d7wZjU8&t=567s>

Eine weitere Arbeit wurde vom Chinesen Peng Zhihui konzipiert. Diese Arbeit hat dabei einen ähnlichen Werdegang wie die vorliegende Arbeit, jedoch deutlich fortgeschrittener. Die Arbeit wurde, auf dem GitHub Account von Peng Zhihui [12] hochgeladen und ist „open source“ verfügbar. Ein deutlicher Unterschied ist, dass sich die Balancierfähigkeit in dieser Arbeit nicht durch das Einlenken des Lenkers ermöglicht wird, sondern durch eine Gyroskop-Ausgleichsschwungscheibe⁵ halten lässt. Der Vorteil hierbei ist es, dass

⁴ Fahrräder mit großer Transportmöglichkeit durch Gepäckträger

⁵ Hierbei handelt es sich um eine im Rahmen verbauten sich rotierende Scheibe, die einen Gyroskopischen Effekt auslöst

der Lenker, dann nur für die Richtungsfahrt genutzt werden kann, was einiges an Ressourcen des Mikrokontrollers spart. Was bei einer solchen Herangehensweise ermöglicht wird, ist eine Balancierfähigkeit im Stand. Denn sobald die Lagesensoren erkennen, dass sich das Fahrrad in einer der jeweiligen Richtung kippt, wird die Ausgleichsschwungscheibe eingesetzt, was zu Folge hat, dass der Lenker nicht mehr für die Balancierung eingesetzt werden muss.

2.3 Mögliche Einsatzgebiete

Wie im Kapitel 2.2 zu lesen ist, sind die Einsatzmöglichkeiten für autonome Fahrräder mit nur zwei Auflagepunkten nicht sehr vielfältig ausgelegt. Da sich dreirädrige Lastenräder durch ihren dritten Auflagepunkt für viele Einsatzgebiete deutlich besser einsetzen lassen, sind die Einsatzgebiete mit dem in dieser Arbeit vorgestellten Fahrrad deutlich eingeschränkter. Die Vorteile von Fahrrädern mit zwei Auflagepunkten sind für die breite Masse nicht direkt ersichtlich. Zum einen sind Fahrräder, im Gegensatz zu Dreirädrigen Fahrrädern, deutlich wendiger und schneller. Die jeweils maximal gemessene Geschwindigkeit, die ein Fahrrad erlangen konnte, betrug im Jahre 2019, 290 km/h [13]. Da Geschwindigkeiten, wie die im Rekord erwähnten, bei nicht perfekten Bedingungen im Normalfall nicht erreicht werden können, sind die Batterien durch ihre Leistung dazu um Stande. Die erwähnten Vorteile wären für eine Eingliederung bei Radmarathons gut einsetzbar. Durch Einsetzen von solchen autonomen Fahrrädern, können Autos, sowie Mofas, als Fortbewegungsmittel für Kameramänner gut ersetzt werden. Diese würden ohne Fahrer mit einer integrierten Kamera das Fahrgeschehen mitverfolgen und gleichzeitig keinen allzu großen Windschatten den Radfahrern verschaffen, wodurch Sie sonst einen Vorteil gehabt hätten. Durch die programmierten Algorithmen würden die autonomen Fahrräder versuchen sich möglichst unauffällig am Fahrgeschehen zu beteiligen und immer einen möglichst großen Abstand zu anderen Fahrern halten. Bei der verbauten Sensorik würde es sich dann um eine Kamera Sensorik halten, die durch einer KI all die genannten Forderungen erfüllen würde.

3 Aufbau, Erweiterungen und Gesamtkonzept

Dieses Kapitel behandelt die am Fahrrad verbauten Komponenten und vermittelt dem Leser die jeweiligen Entscheidungen bei der Wahl von bestimmten Bauteilen oder Vorgehensweisen. Dabei werden schon verbaute Bauteile, nur in Kürze erwähnt und auf die vorherige Bachelorarbeit [7] verwiesen.

3.1 Modul und Schaltplan Übersicht

Für eine gute Übersicht über die zu verbauende Schaltung, ist die Erstellung eines Schaltplans der erste Schritt. Durch ein solches Simulationsprogramm können genutzte Schaltungen simuliert und vorab getestet werden. Ein solches Simulationsprogramm hilft dabei, im Voraus zu wissen, wie die Schaltung auf verschiedene Eingangssignale oder Spannungen reagiert und mit welchen Ausgangsspannungen zu rechnen ist. Die Wahl für das Programm fiel hierbei auf LTSpice, da dieses Programm besonders gut für die Simulation von Schaltnetzen geeignet ist. Dabei lassen sich die Zeitverläufe darstellen und berechnen, die sich beim Verbauen weiterer Bauteile als sehr hilfreich erwiesen haben. Dabei können Parametervariationen durchgeführt werden, um sich bei Veränderungen des Systems ein Bild über die aktuelle Situation des System zu verschaffen. Dadurch ist es möglich, dementsprechende Anpassungen am System vorzunehmen.

In unserem Fall wurde eine Modellierung des gesamten Systems bis in jede unten erwähnte Komponente beschrieben und vorgestellt. Das Herzstück unserer Schaltung ist der BBB der mit verschiedenen Signalprotokollen die verschiedenen Bauteile anspricht. Diese Protokolle konnten durch Pulse-Signale⁶ simuliert werden, um bestimmte Bitfolgen an Datenbytes darzustellen. Eine genauere Erklärung des Einsatzes ist im Kapitel 3.3 zu finden. Die Spannungsversorgung erhält das Fahrrad durch zwei 24V Batterien, welche über zwei verschiedenen Notaus-Schaltern abgesichert sind. Der erste der beiden Notaus-Schalter schließt die komplette 48V Spannungszufuhr, wobei der zweite Schalter nur die 24V für das System ohne den Hinterradantrieb ausschaltet. Im Laufe der Bearbeitungszeit wurde ein dritter Notaus-Schalter hinzugefügt, um die Spannungszufuhr für den Lenkmotor abzdrehen. Man entschied sich für diesen Weg, da es bei Codeabbrüchen zu keinem Herunterfahren des Systems kam, was dazu führte, dass sich der Lenker ungebremst in eine Richtung drehte und die verschiedenen Kabel in Mitleidenschaft zog.

Das Durchdrehen des Lenkers hatte drei Gründe: Zum einem kam es durch das Abfragen des Encoders zu Ausfällen, sobald ein Antworten vom Encoder erwartet wurde, wobei dieser aus unerklärlichen Gründen keine Antwort lieferte. Dieses Problem wurde durch eine fail-exit⁷ Funktion umgegangen, die dabei die Anzahl der fehlgeschlagenen Abfragen mitzählt und bei einer bestimmten Anzahl an Abfragefehlern das Programm abbricht. Nach dem Einfügen dieser fail-exit Strategie kam es darauf zu keinen Encoder Ausfällen mehr, was darauf hindeutete, dass die O_NONBLOCK⁸ Option dazu geführt hat, dass dem Encoder keine eine Verzögerung erlaubt worden war.

⁶ Als Pulse-Signal sind Spannungsverläufe einer Spannungsquelle zu verstehen, die im LTSpice Programm eingestellt werden können.

⁷ Ein im Code befindliche Funktion, die bei bestimmten Situationen, den Code abbricht und die System runterfährt.

⁸ Eine in der open() Funktion, einzutragender Parameter, die bei Lesezugriffen nicht auf eine Antwort wartet.

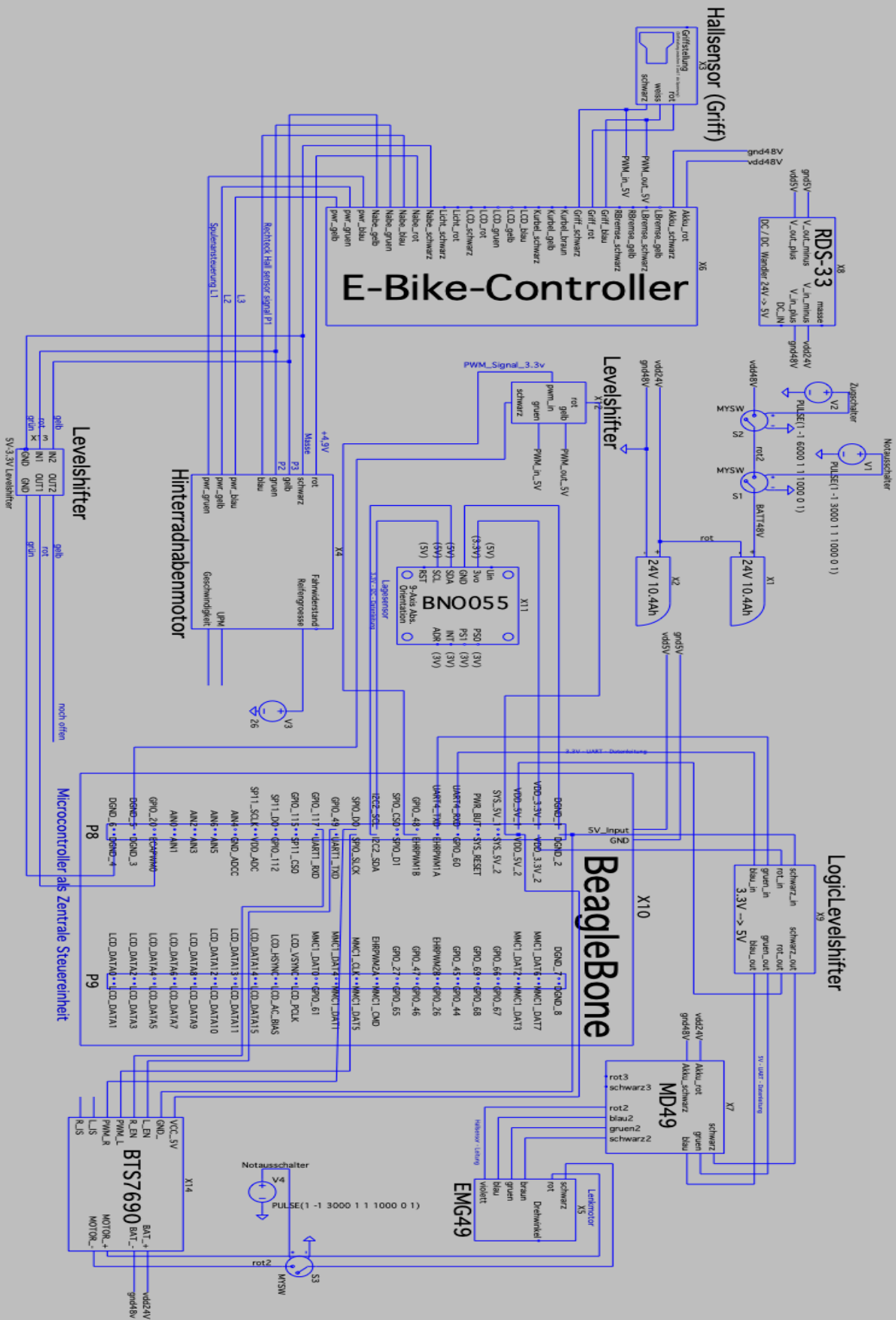


Abbildung 4: Übersicht aller verbauten Komponenten (LTSpice)

Ein weiterer Grund für einen System Ausfall war ein Überschreiben des Ausgabearrays⁹. Durch das Vollschieben des Arrays konnte der Encoder nicht mehr ausgelesen werden, wodurch es zu einem segmentation-fault¹⁰ gekommen war. Dadurch konnte es zu einem Ausschalten des PWM Signales nicht mehr kommen und der Lenker ist dann wiederum durchgedreht. Dieser Fehler wurde durch das Setzen eines nicht erreichbaren statischen Zielwertes beseitigt.

3.2 Komponenten

Die verbauten Komponenten für eine Balanceregung wurden nach dem nachfolgenden Prinzip ausgesucht: Beschleunigen, Abfragen des Winkels, anzufahrenden Lenkwinkel bestimmen und Lenkwinkel anfahren. Durch diesen Ablauf in jedem der Regeldurchgänge kann das Fahrrad seine Balance halten und sich selbst auskurieren.

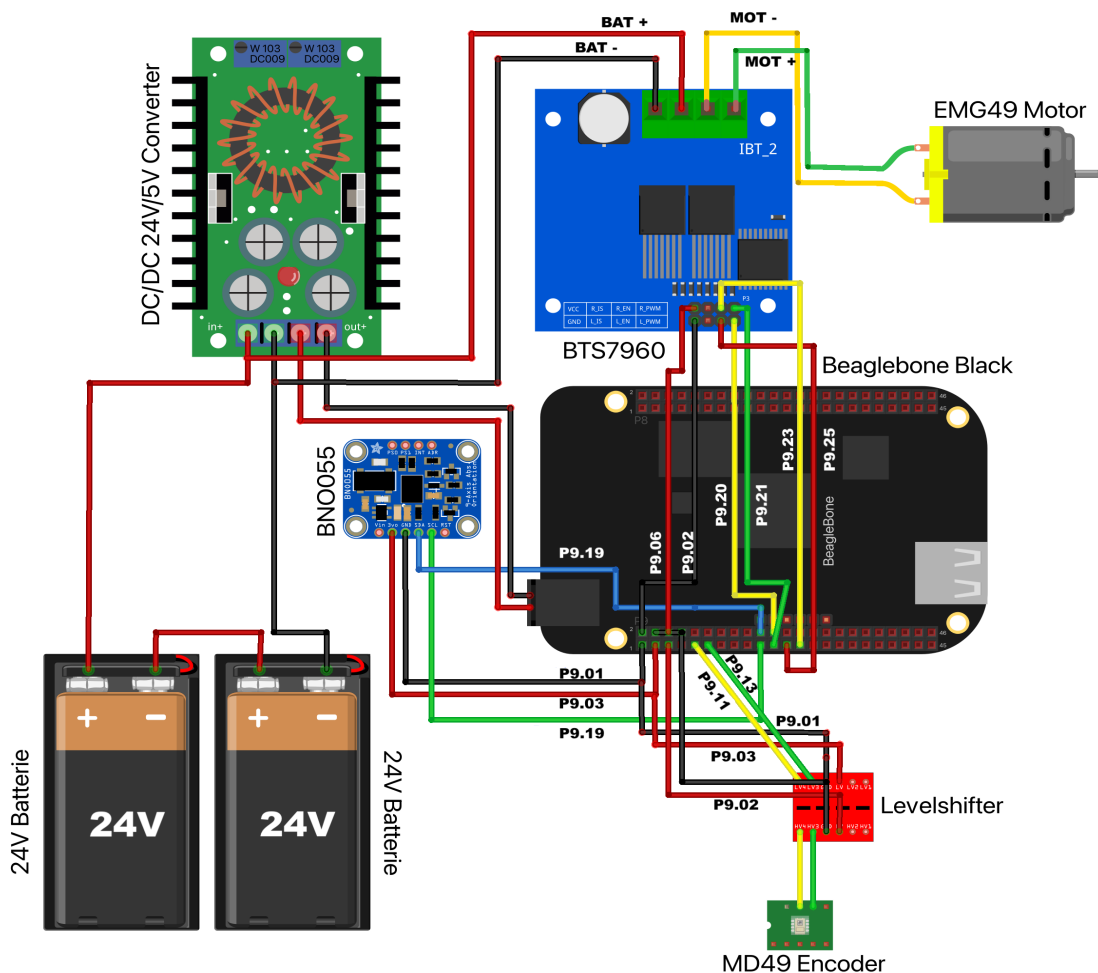


Abbildung 5: Übersicht der verbauten Komponenten ohne Antriebsmotor (Fritzing)

⁹ Ein Speichermedium das für Schreib und Lesezugriffe nur wenige Bytes benötigt.

¹⁰ Eine Zugriffsverletzung, die bei Zugriff auf nicht erreichbaren Daten eintritt.

3.2.1 BNO055¹¹ Lagesensor

Der BNO055 Lagesensor ist für das kontinuierliche Abfragen der drei Eulerwinkel¹² zuständig. Für diese Arbeit sind dabei nur zwei der drei Eulerwinkel von Bedeutung. Dabei bestimmt der Rollwinkel das Kippverhalten des Fahrrads, wodurch Rückschlüsse über die aktuelle Lage des Fahrrades gezogen werden können. Darüber hinaus ist der Sensor dazu in der Lage, die Sensordaten vom Beschleunigungsmesser, dem Gyroskop¹³ und dem Magnetometer abzufragen und für bestimmte Anwendungsgebiete zu nutzen. Um nun auf den Wert des Rollwinkels zuzugreifen, müssen wir zu Beginn eine Abfolge von Funktionsaufrufen von BBB durchführen. Diese Funktionsaufrufe sind dabei Folgende:

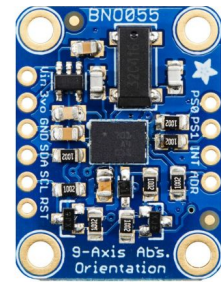


Abbildung 6:
BNO055 Lagesensor

Quelle: <http://fab.cba.mit.edu/classes/863.19/EECS/people/bibekpandit/week9.htm>

1. **get_i2c():** Zu Beginn stellt man eine Verbindung zum BNO055 über eine Inter-Integrated Circuit (I2C) Schnittstelle her. Dabei werden die Pins P9.19 und P9.20 genutzt, die zum einem das I2C Protokoll zu unterstützen. Der Pin P9.19 ist dabei ein SCL („Serial clock“) und gibt dem Takt der Datenübertragung an, welche für die Übertragungsgeschwindigkeit zuständig ist. Der Pin P9.20 ist ein SDA Pin („Serial Data“), die dabei zur Datenübertragung dient. Um sich mit dem BNO055 zu verständigen, muss der im Verzeichnis „/dev/bus/i2c“¹⁴ befindliche „I2C-state“ aktiviert werden. Dadurch wird eine Verbindung zum BBB ermöglicht.
2. **set_page0():** Durch diesen Funktionsaufruf im Code wird ein Verweisen auf die zweite Seite der Register des BNO055, durch das Senden von 2 Bytes gesetzt. Das erste Byte übermittelt dem BNO055, dass die Seite gewechselt wird wobei im zweiten Byte die jeweilige Zeile angegeben wird.
3. **set_axis_mode():** Darauffolgend wird übergeben, welcher Achsenmodus genutzt werden soll. Hierbei werden die Eulerwinkel genutzt, und dem BNO055 über eine Bitfolge dieses übermittelt. Bei den Eulerwinkel handelt es sich um drei Winkel, die zum Beschreiben eines Körpers im dreidimensionalen Raum dienen.
4. **set_mode():** Hierbei wird der Fusion-modus eingestellt. Der gewählte Modus ist der IMU-Modus, der die Beschleunigungssignale sowie Gyroskop Signal vereint. Dadurch erhalten wir, aus diesen beiden Signalen den benötigten Eulerwinkel.
5. **get_roll():** Zum Schluss wird der gebrauchte Eulerwinkel in der Regelschleife kontinuierlich abgefragt. Hierbei handelt es sich um den Rollwinkel für das Abfragen des Kippverhaltens. Um sich ein Bild über die Schräglage des Fahrrades machen zu können, wird der Rollwinkel dabei abgefragt und dem Regelerverlauf weitergeleitet.

¹¹ Bezeichnung des Lagesensors.

¹² Ist ein Zusammstellung von drei Winkeln, mit denen eine Orientierung eines Körpers im dreidimensionalen Raum beschreibbar ist.

¹³ Zur Messung von Drehbewegungen einzusetzen.

¹⁴ Der Dateipfad des BBB für die Umstellung einzelner Modis der jeweiligen Pins.

Der Yawwinkel¹⁵ wird im Rahmen dieser Arbeit nicht explizit genutzt. Trotzdem ist ein Konzept im Code vorhanden. Dabei wurde versucht mittels dem Yawwinkel einen weiteren Eulerwinkel miteinzubeziehen, der dabei die Fahrtrichtung des Fahrrads wiedergibt. Hierdurch sollen Abweichungen einer gezielten Fahrtrichtung korrigiert werden, um dadurch eine möglichst gerade Fahrweise sicherstellen zu können. Abweichungen einer solchen geraden Fahrt, werden durch regelmäßiges Auskurieren eines Kippens in nur eine Richtung hervorgerufen. Dabei kann es sich um eine schlecht ausgebaute Fahrstrecke oder ähnliches handeln.

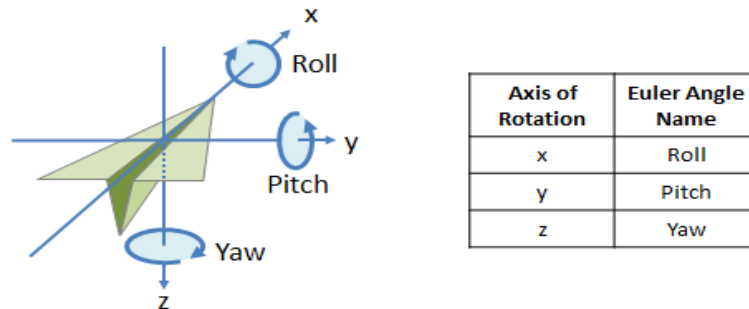


Abbildung 7: Übersicht über Eulerwinkel

In Anlehnung an:
<https://deacademic.com/dic.nsf/dewiki/1196043>

Die gemessenen Eulerwinkel werden dann im Anschluss an den BBB übergeben, sodass dieser den Wert transformiert und in der Regelung weiter transportiert.

3.2.2 MD49 Encoder

Der MD49 wurde für den Einsatz für den Lenkmotor (EMG49) entwickelt und ist in der Lage zwei Motoren unabhängig voneinander anzutreiben. Hierfür ist eine Versorgungsspannung von 24V für einen der Motoren nötig. Es ist möglich über verschiedene Kommandos bestimmte Aktionen durchzuführen oder Informationen über dem Motor abzufragen. Hierbei kann der aktuelle Stromverbrauch, der aktuelle Encoder Wert sowie das aktuelle Beschleunigungslevel zurückgegeben werden. All diese Informationen können über die Hallensoren des Motors, die am MD49 angeschlossen sind, abgefragt werden. In unserem Fall nutzen wir folgende Funktionsaufrufe:



Abbildung 8: MD49 Motorsteuerung

Quelle: <https://www.roboterbausatz.de/p/dc-motortreiber-h-bruecke-bts7960-5-5v-bis-27v-43a-pwm>

1. **set_up_uart():** Zu Beginn stellt man eine Verbindung zum MD49, über eine der Universal Asynchronous Receiver / Transmitter (UART) Schnittstelle her. Dazu verweist man auf die Pins P9.11 und P9.13 und gibt dem MD49, damit zu verstehen, dass diese Pins die Schnittstelle zum BBB sind. Hierbei handelt es sich bei dem Pin P9.11, um einen Receive Data (Rx)D Schnittstellenleitung und bei dem Pin P9.13, um eine Transmit Data (Tx)D Schnittstellenleitung zum Empfangen sowie Versenden von Daten.

¹⁵ Ein Eulerwinkel der zu Bestimmung der Fahrtrichtung eingesetzt werden kann.

2. **reset_MD49_encoder():** Mit dieser Funktion wird ein Zurücksetzen des Encoders auf den Null Wert ermöglicht. Dadurch orientiert sich der Motor wieder neu, und setzt seine aktuelle Position als seinen globalen Nullwert zurück. Dabei handelt es sich um einen Wertebereich von 0 bis 980, die eine komplette Rotation widerspiegeln soll. Nach einer solchen Rotation wird nicht wie üblich bei dem Startwert begonnen, sondern es kommt zum Weiterzählen der Encoder Ticks.
3. **get_MD49_encoder():** Hierdurch wird der aktuelle Encoder Wert über die Hallsensoren¹⁶ des EMG49 an den MD49 übergeben, um einen Orientierungspunkt des Motors zu erhalten. Dieser gibt dann die aktuelle Lenkerposition des Fahrrades an, wodurch über festgehaltenen Rohdaten zurückzuverfolgen werden kann, wie sich der Lenker im Laufe einer Fahrt verhalten hat.

Acceleration register	Time/step	Current speed	New speed	Steps	Acceleration time
1	16ms	0	255	255	4.08s
2	16ms	127	255	64	1.024s
3	16ms	80	0	27	0.432s
5 (default)	16ms	0	255	51	0.816s
10	16ms	255	0	26	0.416s

Abbildung 9: Tabelle für Beschleunigungslevel des EMG49 [14]

Zu Beginn der Arbeit wurde diese Motorsteuerung auch als einzige Komponente für die Steuerung des EMG49 genutzt. Im Laufe der Zeit wurde die Steuerung des Motors an die zweite Motorsteuerung (BTS7960) übergeben da die Reaktionszeit, durch die intelligente Motorschutzfunktion gebremst wurde. Dabei konnte ein maximaler Beschleunigungswechsel von 0.416 Sekunden erbracht werden, was für den Einsatzzweck eine zu hohe Beschleunigungszeit darstellte. Da es sich in dem Anwendungsfall, um kleine Einlenkverfahren zum Auskurieren handelten, wurde aus diesem Grund die Steuerung des EMG49, dem BTS7960 überlassen. Dabei wurde der MD49 nur noch für Encoder Abfragen genutzt, da sich dieser am besten dafür einsetzen ließ. Aufnahmen zu den Verschiedenen Beschleunigungslevel, sind unter dem Verzeichnis (Aufnahmen/Lenkmotor/Alte Motorsteuerung/Beschleunigung) zu finden.

¹⁶ Dabei handelt es sich um passive Sensoren die Spannungsunterschiede messen, die am Leiter durch Magnetfelder erzeugt werden.

3.2.3 BTS7960 Motorsteuerung

Der BTS7960 ist eine sehr simple Motorsteuerung, welche durch zwei PWM Eingänge seine Bewegungsrichtung, sowie Beschleunigung regelt. Die doppelte H-Brücke¹⁷ kann den Motor daher auf einen maximalen Beschleunigungswechsel von ca. 0.04299 Sekunden beschleunigen.

Ein höheres Beschleunigungswechsel wäre theoretisch mit dem BTS7960 möglich, aber um den EMG49 bei Testfahrten nicht zu stark zu beanspruchen wurde sich dagegen entschieden. In den Aufnahmen zum Motor ist dabei der Unterschied des Beschleunigungswechsel gut erkennbar

(siehe Bachelorarbeit/Aufnahmen/Lenkmotor/Neue_Motorsteuerung). Dabei wird eine Versorgungsspannung von 5.5V bis 27V genutzt welche sich in diesem Fall bei einer Spannung von 24V befindet. Die dabei verwendete H-Brücke nutzt die an der BTS7960 angebrachten Schalteingänge, um eine Fahrtrichtung zu bestimmen. Sobald eines der beiden PWM Eingänge betrieben wird, kommt es dazu, dass einer der beiden Strecken in der verbauten H-Brücke genutzt wird. Dadurch lässt sich der Motor in jeweils eine der beiden Richtung drehen. Die dabei genutzte Geschwindigkeit, werden über den Tastgrad¹⁸ des PWM Signals bestimmt, die den Motor dann mit seiner Maximalspannung von 24V betreiben lässt. Mit den beiden Pins „R_IS“ und „L_IS“ kann daraufhin die hierbei genutzte Stromstärke ausgelesen werden, wodurch Auskunft über die aktuelle Geschwindigkeit gegeben werden kann. Diese Funktionalität wurde in diese Arbeit, durch die im Kapitel 3.5.1 vorgestellte Bibliothek ermöglicht, sodass auf diese Pins nicht zugegriffen werden musste.

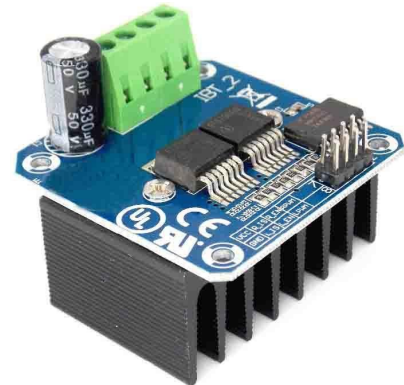


Abbildung 10: BTS7960 [18]

3.2.4 Pegelumsetzer (Levelshifter)

Der von uns genutzte Pegelumsetzer wurde für eine Verstärkung des UART Signals zwischen dem BBB und dem MD49 genutzt. Hierbei wurde die Signalspannung von 3.3V auf 5V verstärkt und konnte damit ein Erkennen des Signals durch den MD49 garantieren. Man unterteilt einen Levelshifter in eine High und Low Seite, an denen sich zum einen eine Spannung von 3.3V, sowie eine Spannung von 5V anliegen. Der genutzte Levelshifter besitzt vier Kanäle auf dieser er bidirektional Signale überträgt. Weitere Levelshifter wurden für das Übertragen des Rechtecksignals des Hinterrades für die Geschwindigkeitsüberwachung genutzt, sowie für die Transformation des PWM Signals in eine Gleichspannung für das Antreiben des Hinterrades.

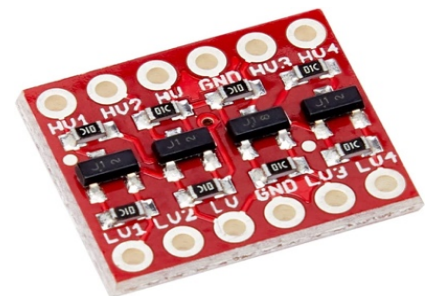


Abbildung 11: Verbauter Levelshifter

Quelle: <https://learn.sparkfun.com/tutorials/bi-directional-logic-level-converter-hookup-guide/all>

¹⁷ Sie dienen zur Ansteuerung von Gleichstrom sowie Schrittmotoren, bei denen die Drehzahl sowie Drehrichtung einstellbar ist

¹⁸ siehe Kapitel 6.4

3.3 Beaglebone Black

Der Beaglebone Black fungiert als zentrales Steuerglied und kommuniziert mit allen verbauten Bauteilen. Der Grund weswegen sich hierbei nicht für eines der anderen Mikrokontroller (wie z.B. den RaspberryPi oder einen Arduino) entschieden wurde, ist die deutliche höhere Anzahl an belegbaren Pins, sowie dem Nutzen einer von der CPU separaten „programmable real-time unit“ (PRU) Einheit, die Echtzeitanwendungen ermöglicht. Da eine geringe Reaktionszeit für ein autonomes Fahrrad von immenser Bedeutung ist, fiel die Wahl auf den BBB. Hierbei werden im Rahmen dieser Arbeit die PWM, PRU, I2C, UART sowie die „General Purpose Input/Output“ (GPIO) Schnittstellen genutzt, die zu Beginn im Verzeichnis „/sys/devices /platform/ocp/“ für die jeweilige Schnittstelle zu belegen ist.

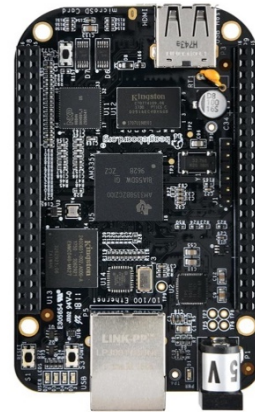


Abbildung 12: Beaglebone Black

Quelle:
<https://www.okdo.com/de/p/beaglebone-black-revision-c/>

Folgende Unterteilung der genutzten Schnittstellen sind:

1. **PWM:** Für die Steuerung der Geschwindigkeit des Hinterradantriebs, sowie die Steuerung des Lenkmotors und der jeweiligen Lenkrichtung durch Nutzung von zwei PWM Ausgängen.
2. **UART:** Für die Kommunikation mit dem EMG49. Hierbei werden die Kommandos mittels des UART Protokolls an die Motorsteuerung übergeben, sowie bestimmte Systeminformation empfangen.
3. **eCAP (PRU):** Für die Kommunikation mit dem PRU Chipsatz und dem Empfangen des aktuellen „Timer Werts“¹⁹ des Motors.
4. **GPIO:** Zur Aktivierung sowie Deaktivierung der Funktionalität des Lenkmotors.

Die Kommunikation mit dem BBB fand zu Beginn dieser Arbeit über eine „Local Area Network“ (LAN)-Verbindung statt. Durch das Bestimmen einer statischen IP-Adresse, war es möglich über eine Secure Shell (SSH), einen sicheren Zugriff auf den BBB zu erhalten. Diese wurde über ein 15 Meter langes LAN-Kabel hergestellt, welches bei Testfahrten meisten wegen seiner Inflexibilität Probleme auslöste. Man entschied sich daher auf eine „Wireless Local Area Network“ (WLAN)-Verbindung zu setzen, die durch das Erstellen eines „Wifi access point“ (WAP) auf dem BBB aufgesetzt wurde. Man installierte einen „host access point daemon, (Hostapd) welcher für das Aufbauen eines Access Points zuständig war. Durch den Hostapd war es nun möglich, unabhängig von einem Network Manager²⁰ einen Access Point aufzubauen und den BBB nun durch ein WLAN-Netzwerk anzusprechen. Die erhöhte Latenz der WLAN-Übertragung, stellte keine Probleme dar, da sich im finalen Code keine ausgeführten Funktionen befanden, die mit dem genutzten Laptop kommunizieren mussten und dadurch alle ausgeführten Operationen auf dem BBB stattfanden. Ausschließlich am Anfang wurden bei Start des

¹⁹ Ein in der PRU integrierter Zähler, der wiederkehrende Ereignisse parallel mitzählt.

²⁰ Ist eine im Betriebssystem eingebaute Systemsoftware, welche für die Verwaltung von Netzwerkverbindungen genutzt wird.

Codes benötigte Parameter übergeben. Dabei handelte es sich um die Übergabe des Parameters für den Pr_Regler²¹, der zufahrenden Zeit und Geschwindigkeit.

3.4 Pulsdauermodulations Verfahren

Bei der Pulsweitenmodulation(PWM) handelt es sich um ein gezieltes Verhältnis des Spannungspegels und der Periodendauer eines Rechtecksignals²², das bei einer festen Grundfrequenz variieren kann [14]. Bei den Spannungspegeln handelt es sich um mögliche Spannungen am BBB von 0V und 3.3V. Dabei ist es möglich Drehgeschwindigkeiten der beiden Motoren zu regeln.. Hierbei nutzt man die Eigenschaft des Motors aus das ein Abtriebsmotor beim Abklemmen von der Spannung in einen Leerlauf gelangt, der ihn langsam zum Stillstand bringt. Um nun den Motor auf halbe Geschwindigkeit bringen zu können, muss für die Hälfte der Zeit eine Spannung von 3.3V und für die andere Hälfte der Zeit eine Spannung von 0V anliegen. Bei einer genügt kleinen Periode des PWM Signals, ist eine Geschwindigkeitszunahme sowie Abnahme dann nicht mehr erkennbar.

Als Beispiel wird eine Periodendauer von 2ms angenommen. Dabei wird für 1 ms eine Spannung von 3.3V, sowie für 1ms eine Spannung von 0V ausgegeben. Dadurch wird in der Theorie eine Drehgeschwindigkeit von 1/2 der Maximalgeschwindigkeit erreicht. Die Periodendauer errechnet sich wie folgt:

$$Periodendauer = \frac{1}{Frequenz}$$

Am Beispiel des BBB wäre eine Frequenz von 1Mhz in der Theorie möglich, nehmen aber eine einzustellende Frequenz von 500Hz an:

$$Periodendauer = \frac{1}{500Hz} = 0.002 \text{ sekunden} = 2 \text{ millisekunden}$$

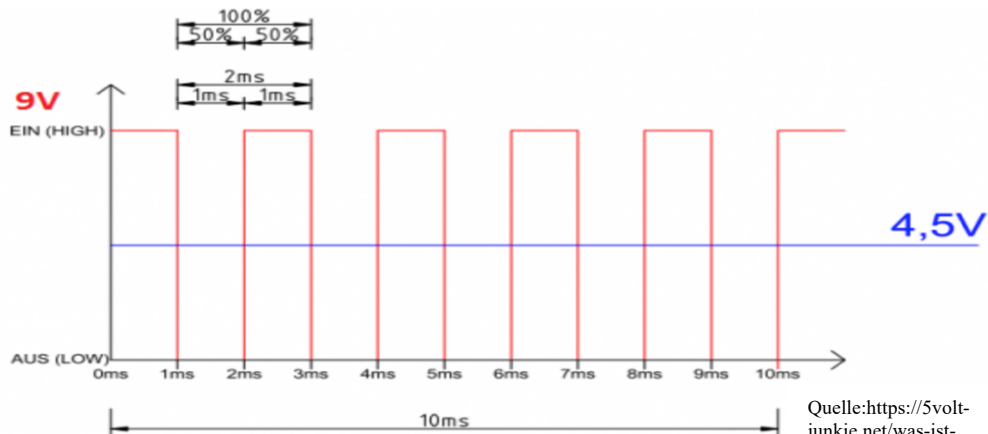
Bei einer Pulsdauer sowie einer Pausenzeit von 1ms, erhält man einen Tastgrad von:

$$Tastgrad = \frac{Pulsdauer}{Periodendauer} = \frac{1ms}{2ms} = 0.5 = 50\%$$

Und somit bekämen wir bei einer Versorgungsspannung von 3.3V einen tatsächliche Endspannung von 1.65V heraus.

²¹ Ein P-Regler, welcher die Umrechnung des Rollwinkels zuständig und bei der Berechnung Soll-Encoder beitrug (genauer in Kapitel 4.3).

²² Periodisches Signal das zwischen zwei Spannungszuständen hin und her schaltet.



Quelle: <https://5volt-junkie.net/was-ist-pwm-ein-kleines-tutorial/>

Abbildung 13: Veranschaulichung eines 50%igen Tastgrad

3.4.1 Antreiben der Motoren

Die Anwendungen, die von der PWM Schnittstelle profitieren, sind die beiden Motoren die für das Betreiben des Lenkmotors sowie des Antriebsmotors gedacht sind.

Der Lenkmotor wird durch zwei Pins am BBB betrieben, die als PWM Pins belegt worden sind. Beide Pins funktionieren komplett unabhängig voneinander und geben dabei die Geschwindigkeit sowie die Fahrtrichtung des Lenkers an. Am BTS7960 sind dabei zwei PWM Eingänge verbaut (L_PWM und R_PWM). Diese beiden Pins sind dabei wiederum durch jeweils zwei weitere Eingänge (L_EN und R_EN) gekoppelt, die die PWM Eingänge durch setzen eines „HIGH“²³, freischalten. Für den Ablauf des Codes, ist es Wichtig, dass zu keinem Zeitpunkt des Regelerdurchlaufs zu Signalen beider PWM Pins kommt, da es dabei dann zu Komplikationen kommen kann.

Der Antrieb des Hinterradmotors wird durch einen verbauten Pegelumwandler ermöglicht, der durch die BBB erzeugten PWM Signale in eine Gleichspannung umwandelt. Hierbei kann durch ein Verstellen des Tastgrades angegeben werden, wie stark die Spannung am Ausgang sein soll und die Geschwindigkeit dadurch regelt. Es sind dann verstärkte Spannungen im Bereich von 0 bis 5V möglich. Die durch die PWM Pins erzeugten 3.3V, werden durch einen Verstärkungsfaktor auf die benötigten 5V verstärkt. Dabei sind Geschwindigkeiten von bis zu gemessenen 60km/h erreichbar [7]. Diese werden für unseren Zweck nur zu einem kleinen Prozentteil ausgenutzt.

Die genutzte Frequenz von 100kHz wurden durch stetiges Erhöhen der Frequenz ermittelt. Da sich die Rechteckspannung des PWM Signals bei einer Frequenz von 1MHz, als recht unsymmetrisch herausstellte (siehe /Bilder/PWM/Oszilloskop), wurde sich dabei für eine Frequenz von 100kHz geeinigt.

In dieser Arbeit ist die Geschwindigkeit des Antriebsmotors ein Richtwert für den Lenkmotor und daher dem Lenkmotor übergeordnet. Dieser könnte sich dann auch komplett unabhängig vom Regelkreis bewegen.

²³ Ein „HIGH“, ist auch als ein Legen einer von zwei möglichen Spannungen (0V und 3.3V) am Pin zu verstehen.

3.5 Programmable real-time unit

Die programmierbare Echtzeiteinheit und das damit einhergehende Kommunikationssystem (PRU-ICSS) ist ein weiterer Prozessor, der eng im IO-Subsystem integriert ist. Dabei wird eine niedrige Latenz bei der Steuerung der IO-Pins²⁴ unterstützt. Diese sind für Echtzeitaufgaben von großer Bedeutung und ermöglichen eine geringe Reaktionszeit. Sie sind in den Modellen AM335x, AM437x, AM57x und AM65x der Sitara-Prozessoren²⁵ verbaut, zu welchen auch der BBB gehört [15]. Die PRU ist dabei vom Benutzer programmierbar und kann zum Implementieren simpler Steuerprogramme genutzt werden. Meist werden diese in Assembler geschrieben, um möglichst hardwarenahe und reaktionsstark sein zu können. Die Benutzung eines solchen PRU-ICSS ist dabei sehr praktisch da somit das Hernehen einer weiteren Steuereinheit gespart werden kann.

3.5.1 eCAP und libpruio Bibliothek

Die libpruio Bibliothek ist ein Treiber für Mikroprozessoren des Typs „TI AM335X“ und wurde für das Arbeiten mit dem BBB entwickelt. Dabei wird die libpruio Software auf den ARM „Hostprozessor“ ausgeführt, welche mit der zuzüglich verbauten PRU kommuniziert. Hierbei wird dann auf den einprogrammierten „Timer“ zugegriffen, welcher auf dem PRU als Subsystem eingesetzt ist. Der eingesetzte „Timer“ wird hierbei als eine Art Geschwindigkeitsmesser genutzt. Die Präferenz für die Geschwindigkeit sind die im Hinterrad verbauten Magnete. Sie stellen den Aufbau eines Hallsensors dar, wodurch ausgelesen werden kann welche Strecke in welcher Zeit zurückgelegt wurde. Die im Hallsensor verbauten Magnete erzeugen bei Drehung des Rades am Ausgang ein Rechtecksignal, welches abwechselnd Spannungen von 0V und 2.5V erzeugen. Bei dem dabei erzeugtem Tastgrad handelt es sich in jedem Fall um einen Wert von 50%, da der Aufbau der Magnete identisch ist. Der einzige Indikator für die Drehgeschwindigkeit des Rades ist die Frequenz, welche sich proportional zur der Drehgeschwindigkeit verhält. Diese wird dann vom BBB am Pin P9.42 als Eingang eingelesen und von der PRU parallel zur Host-CPU mitgezählt.



Abbildung 14: Abbildung der Magneten im Antriebsmotor

Quelle: <https://de.aliexpress.com/item/1005002474276191.html?gatewayAdapt=glo2de>
...

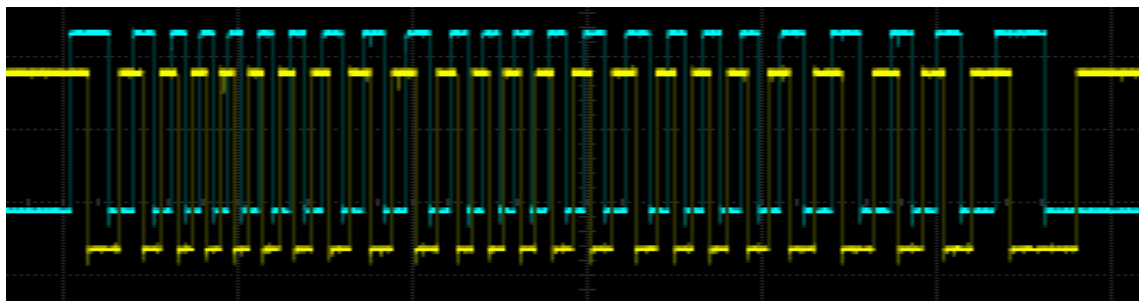


Abbildung 15: PWM Ausgang des Hinterradantriebs (Ausgabe des Oszilloskop)

²⁴ Pins am BBB die jeweils als Eingänge sowie Ausgänge genutzt werden können.

²⁵ Prozessoren der Marke Texas Instruments.

Bei der Herleitung der Geschwindigkeit aus der Frequenz, muss die Anzahl der im Motor verbauten Magneten bekannt sein. In diesem Fall handelt es sich um 46 Magnete, welche für die abwechselnden Spannungen verantwortlich sind. In der *Abbildung 14* ist dabei eine komplette Umdrehung des Rades zu erkennen, wodurch 23 Perioden erzeugt wurden. Der Umfang des Hinterrades entspricht dabei 0.414m. Dabei entsprechen:

$$\begin{aligned} 23 \text{ Perioden} &\triangleq 0.414\text{m} \\ 1 \text{ Periode} &\triangleq 0.018 \text{ m} \end{aligned}$$

Hierdurch ist nun bekannt, welche Distanz, eine Periode des Hallsensors entspricht und welche Distanz dabei zurückgelegt wird. Die dabei gezählten Perioden, werden zusammengezählt und zurückgegeben. Dadurch kann bei beispielsweise 430 Perioden, ausgesagt werden, dass:

$$430 \text{ Perioden} \triangleq 7.74 \text{ m/s} \triangleq 27.864 \text{ km/h}$$

Durch diese Umrechnungen kann im Anschluss, Auskunft über die aktuelle Geschwindigkeit des Fahrrades gegeben werden.

3.6 Anpassung der Stützräder

In diesem Kapitel wird versucht einen für den Einsatzzweck passenden Winkel der Stützräder zu ermitteln. Bei einer Geschwindigkeit von mehr als 0 km/h muss sich das Fahrrad neigen, um dabei um eine Kurve kommen zu können. Durch den Ausgleich der Fliehkraft²⁶ mit der Gewichtskraft²⁷ kann das Fahrrad letzten Endes die Bahn in der Kurve halten und bricht nicht aus dieser aus.

Würde das Fahrrad exakt senkrecht stehen, würde es in der Theorie auch nicht zu einem umkippen führen. Aber schon bei den kleinsten Außeneinwirkungen kippt das Fahrrad um und es kann dadurch zu Beschädigungen der Elektronik kommen.

Bei exakter senkrechter Aufrichtung zeigt sich jedoch, dass die Gewichtskraft vom Schwerpunkt auf den Radaufstandspunkt liegt. Dies ist wie in *Abbildung 16* zu sehen. Dabei ist zu erkennen, dass die Gewichtskraft F_G durch die Erdbeschleunigungskonstante g und der Masse m des Fahrrads wie folgt zu ermitteln ist:

$$F_G = m * g$$

Nun haben wir in unserem Fall nicht nur die optimale Begebenheit, es kommt außerdem zu einer Beschleunigung des Fahrrads, die dazu führt, dass sich das Fahrrad in einer Kurve neigen muss. Dabei hängt der Neigungswinkel von der Kurvengeschwindigkeit ab, bei der sich Fliehkräfte ergeben, welche an das Äußere der Kurve gerichtet sind und der Schräglage entgegenwirken. Dabei ist die Fliehkraft von drei Dingen abhängig: dem Kurvenradius R , der Masse m und der Kurvengeschwindigkeit v . Hierbei ergibt sich folgende Formel:

$$F_{Flieh} = m * \frac{v^2}{R}$$

Man sieht, dass die Geschwindigkeit quadratisch in die Fliehkraft eingeht und wie sehr diese davon abhängt.

Das Fahrrad das bei einem Winkel ϕ , eine Gewichtskraft erzeugt und das Fahrrad umkippen lässt, wird durch einen Sinusanteil der Gewichtskraft, in der Schräglage gehalten, welcher der Fliehkraft entgegenwirkt:

$$F_{Kipp} = m * g * \sin(\phi)$$

Dabei gilt:

$$F_{Kipp} = F_{Flieh}$$



Abbildung 16: Motorrad ohne Schräglage

Quelle: <https://www.cbcity.de/schraeglage-bei-einem-motorrad>

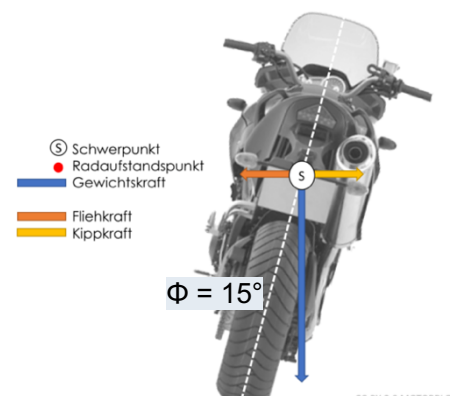


Abbildung 17: Motorrad in Schräglage

Quelle: <https://www.cbcity.de/schraeglage-bei-einem-motorrad>

²⁶ Eine durch Kreisbewegung verursachte Kraft, die von der Rotationsachse weggerichtet ist.

²⁷ Die Kraft die ein Körper in Richtung des Bodens erzeugt.

Für die richtige Wahl des Winkels der Stützräder ist die maximale Schräglage des Fahrrads zu bestimmen. Diese ist dabei unabhängig von der Masse und somit unabhängig vom Passagier [17].

$$\phi = \tan^{-1}\left(\frac{v^2}{R * g}\right)$$

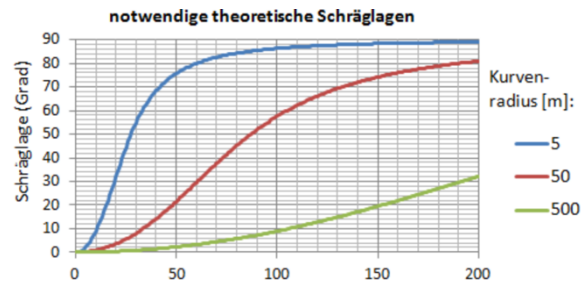


Abbildung 18: Beispiel der verschiedenen Schräglagen

Quelle: <https://www.cbcity.de/schraeglage-bei-einem-motorrad>

Daher ist es auch unwichtig, ob von einem Fahrrad oder einem Motorrad die Rede ist, da die Masse beider nicht ins Gewicht fällt. Dabei sind ihre physikalischen Aufbauten nahezu identisch, sodass von ein und demselben Fahrzeug gesprochen werden kann. Dabei sieht man anhand *Abbildung 18*, dass bei immer höher werdenden Geschwindigkeiten, der Winkel der Schräglage antiproportional abnimmt, und an 90° approximiert. An unserem Beispiel angewendet wird eine Mindestgeschwindigkeit von $4 \text{ km/h} = 1.1 \text{ m/s}$ empfohlen [7], sodass die Lenksteuerung zu genüge eingreifen kann und eine maximale Geschwindigkeit von $20 \text{ km/h} = 5.55 \text{ m/s}$ zu erreichen ist. Dabei beträgt der Durchmesser der Teststrecke am Campus Bockenheim

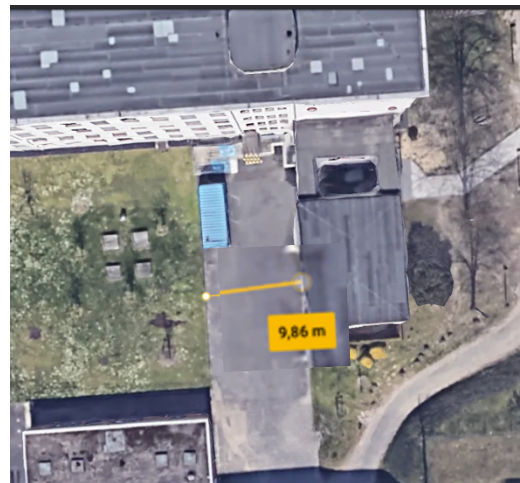


Abbildung 19: Durchmesser der Teststrecke (Google Earth)

(siehe *Abbildung 19*) nach Abmessungen mit Google Earth 9.86 m, sodass der Radius der Strecke 4.93m beträgt. In die Formel eintragen ergibt dies einen Winkel von:

$$\phi_{min} = \tan^{-1}\left(\frac{1.1^2 \text{ m/s}}{4.93 \text{ m} * 9.81 \text{ m/s}^2}\right) = 1.43^\circ$$

$$\phi_{max} = \tan^{-1}\left(\frac{5.55^2 \text{ m/s}}{4.93 \text{ m} * 9.81 \text{ m/s}^2}\right) = 32.49^\circ$$

Nach den gezeigten Berechnungen orientieren wir uns an ϕ_{max} und stellen die Stützräder in einem Winkel von 32.49° ein, um den Ausnahmefall abdecken zu können.

4 Experimente

In diesem Kapitel wurde durch verschiedene Testfahrten, Rückschlüsse auf die Tauglichkeit der jeweiligen Regler gezogen. Um nun zu verstehen, wie die jeweiligen Regler funktionieren und diese aufgebaut sind, behandelt diese Arbeit kurz die Grundlagen der Regelungstechnik. Die Aufgabe eines Reglers ist es die Regelgröße²⁸ abzufragen und diese mit dem erhaltenen Soll-Wert zu vergleichen. Für die zweite Regelung bedeutet dies, dass der vom BBB vorgegebene Soll-Wert eine Regeldifferenz²⁹ durch einen der drei Regler ausgibt und aussagt, wie stark sich diese voneinander unterscheiden. Nun ist es die Aufgabe, die Stellgröße³⁰ insoweit zu verstellen, dass sich der Soll-Wert sowie der Ist-Wert angleichen oder sich zumindest möglichst stark annähern. Der in dieser Arbeit genutzte Regler-Typ, ist der P-Regler. Dabei ist es die Aufgabe des P-Reglers, die mittels einer Verstärkung der Regelabweichung, diese zu vermindern. Bei P-Reglern gilt das Problem, dass sich sehr kleinen Regelabweichungen, nie zu 100% ausgleichen lassen. Hilfe wird dabei meist, mittels eines I-Anteiles hergeholt. Da sich die vorliegende Arbeit nur auf P-Regler beschränkt, werden andere Regelertypen nicht behandelt.

²⁸ Die Ausgangsgröße der Regelstrecke.

²⁹ Die Differenz zwischen dem Soll-Wert und dem Ist-Wert. Das Ziel ist es die Differenz möglich gering zu halten.

4.1 Versuchsaufbau

Der Versuchsaufbau für die Testfahrten ist in folgenden Schritten aufgeteilt. Es wurden verschiedene Zwischenziele angesetzt und in mehreren Testfahrten überprüft, ob eingebaute Bauteile und einprogrammierte Regelungen funktionieren und ob das Fahrrad einigermaßen balanciert. Hierbei war es von großer Bedeutung, die Fahr Voraussetzungen stets konstant zu halten. Da es sich bei der Teststrecke um keine gerade und ebene Strecke handelt, erwies sich das Vorhaben als recht kompliziert und nicht komplett umsetzbar. Bei genutzter Teststrecke handelt es sich um den Hinterhof des Informatikgebäudes der Universität. Diese besitzt eine Größe von ca. 9.5m x 40.5m. Im unteren Drittel der Strecke waren eine Vielzahl von Schlaglöchern und Unebenheiten vorhanden, weswegen eine Nutzung des Fahrrades auf dieser Strecke nicht umsetzbar war und sich daraus nur noch eine fahrbare Teststrecke von ca. 9.5m x 26.5m ergeben hat. Eine weitere Erschwernis war die Steigung hin zum Informatikgebäude von geschätzten ca. 5%. Diese bereits geringe Steigung führte zu einer Geschwindigkeitsverminderung von ca. 40% (siehe Kapitel 5.4). Bei Testfahrten von angesetzten 10km/h wurden daher bei nicht geregelter Geschwindigkeitskontrolle, eine Geschwindigkeit von nur 6km/h erreicht.

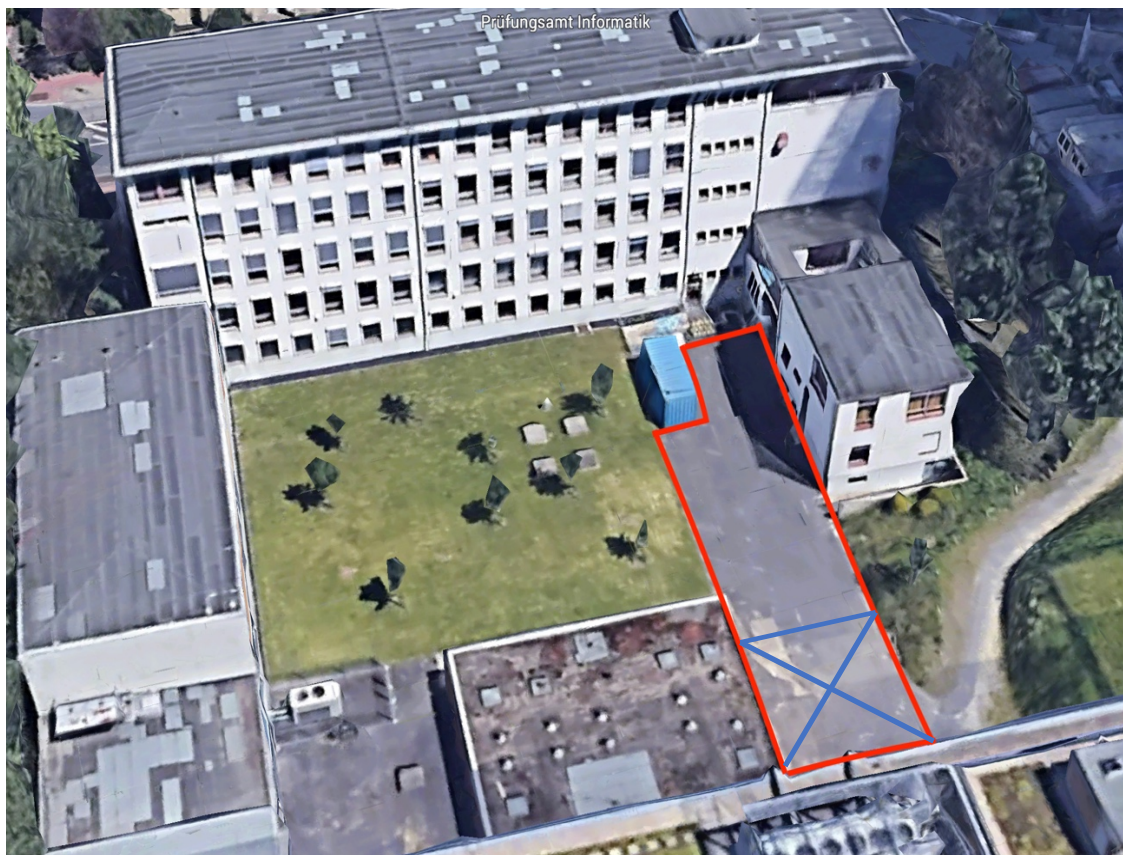


Abbildung 20: GoogleEarth Aufnahme der Teststrecke (Google Earth)

4.2 Optimierung

Die Optimierung des Regelkreises bezog sich in dieser Arbeit, hauptsächlich auf die Reduzierung der Transportzeit³¹ des Reglers. Hierbei waren die Testfahrten, ein guter Weg für das Erkennen von Reglerproblemen. Zu Beginn errechnete man die Transportzeit vor der Optimierung, welche in *Abbildung 21* mit den gemessenen Laufzeiten zu lesen ist.

	Tastgrad setzen	Abfragen Rollwinkel	Abfragen Yawwinkel	Abfragen Ist-Encoder	Transportzeit	Transportzeit + Yawwinkel
Longest time	1.658ms	2.681ms	2.353ms	4.024ms	8.381ms	8.6163ms
Shortest time	0.854ms	1.167ms	1.154ms	3.965ms	5.986ms	6.1014ms
Time difference	0.804ms	1.514ms	1.199ms	0.059ms	2.395ms	2.5149ms
Average time	1.256ms	1.924ms	1.7535ms	3.9945ms	7.1835ms	7.35885ms

Abbildung 21: Laufzeiten einzelner Komponenten

Zu erkennen ist, dass der größte Anteil der Laufzeit, das Abfragen des Ist-Encoders mit maximal ca. 4ms eingenommen hat und dieser auch gleichzeitig die konstanteste Abfragezeit besitzt. Die kleinste Laufzeit hat mit maximal ca. 1.6ms, das Setzen des Tastgrades für den Lenkantrieb. Die mit Abstand größte Zeit Differenz und die damit einhergehende Inkonsistenz besitzt die Abfrage des Yawwinkels, welche nur alle Zehn Regeldurchläufe abgefragt wird, weshalb sich in der Tabelle zwei Transportzeiten zu erkennen sind.

Das Ziel für diese Arbeit ist es zum einen, diese Transportzeit auf mindestens 5ms zu reduzieren und dabei verschiedene Ansätze auszuprobieren. Der erste Ansatz war das Nutzen von Multithreading. Hierbei werden parallel mehrere Prozesse im Code gleichzeitig auf sogenannte Threads³² ausgeführt. Dabei wird versucht, die sequenzielle Ausführung des Codes und das damit einhergehende Abwarten der Ausführungsfolge, zu umgehen und durch eine parallele Ausführung die Transportzeit auf die dabei am längsten andauernde Laufzeit zu reduzieren. Hierbei würde sich die Laufzeit in der Theorie auf ca. 4ms reduzieren, was die größte anteilige Laufzeit der Transportzeit wäre. Leider ließ sich dieses Unterfangen praktisch nicht umsetzen, da es sich bei dem im BBB verbauten Chip um einen Singlecore Prozessor³³ handelt. Trotz allem war ein Effekt an der Transportzeit zu erkennen. Die Differenz der unterschiedlichen Transportzeiten konnte deutlich verkleinert werden, was zu deutlich konstanteren Transportzeiten führte. Ein weiterer Punkt, der zu einer großen Transportzeit führte, war das Abfragen der Encoderwerte. Der BBB sendet dem MD49 mit einem bestimmten Kommando die Anfrage für den Integer³⁴ Wert der aktuellen Encoderstellung. Dabei wartet der BBB auf eine Antwort, bis er diese erhalten hat. Anhand der Zeitmessungen in *Abbildung 21*, ist zu erkennen, dass sich das Warten bis zu 4ms hinziehen kann und dieses nun zu umgehen ist. Beim Öffnen des Verzeichnisses des UART Kommunikation Busses muss daher explizit erwähnt werden, dass nicht auf den Encoder Wert gewartet werden darf. Der dabei erhaltene Encoderwert besteht aus vier Datenbytes, die nun durch den BBB auch in Teilen erhalten werden kann. Dieser kann bei Anfrage des Encoder Wertes mehrere Byte abfragen, bis dieser die kompletten vier Bytes erhalten hat. Hierdurch reduzierte sich ebenfalls die Transportzeit um 75%, wodurch eine Unterschreitung der von uns gesetzten unteren Grenze von 5ms erfolgte.

³¹ Die Laufzeit für einen kompletten Regeldurchlauf.

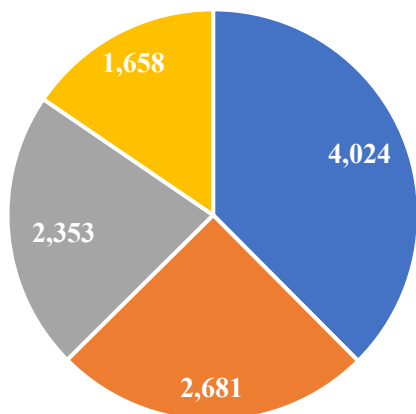
³² Ist ein Ausführungsstrang bei der Abarbeitung eines Programms.

³³ Ein Prozessor der nur einen Prozessorkern besitzt.

³⁴ Ein Datentyp der eine ganzzahligen Wert speichert.

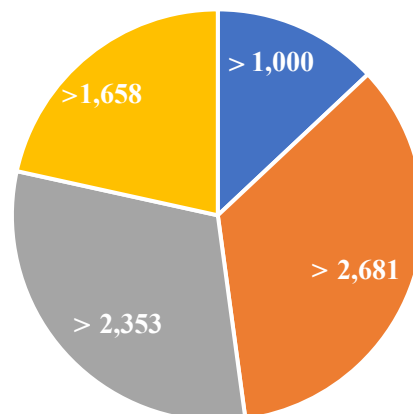
Die daraus resultierende Verteilung der Transportzeiten konnte leider nicht ermittelt werden, da es bei Abfragen des Encoderwertes zu unerklärlichen Fehlern kam. Ein zuzüglicher Effekt der Reduzierung der Abfragezeit des Encoders ist, dass sich die Laufzeiten der drei restlichen Funktion auch reduziert haben müssten, da die Transportzeit hierbei auch unter ihrer einzelnen Laufzeit liegt.

Transportzeit vor der Optimierung



■ Encoder Abfrage ■ Rollwinkel Abfrage
■ Yawwinkel Abfrage ■ Tastgrad setzen

Transportzeit nach der Optimierung



■ Encoder Abfrage ■ Rollwinkel Abfrage
■ Yawwinkel Abfrage ■ Tastgrad setzen

Ein weiterer Punkt für die Optimierung des Systems ist die Vermeidung von Systemausfällen. Ein Grund für Systemausfälle waren die zu locker anliegenden Bauteile, welche in ihren Steckerleisten verbaut waren. Diese sind bei Testfahrten durch Vibrationen oder ähnlichem von ihren Halterungen heruntergekommen und führten dazu, dass der BBB keine weiteren Daten von diesen erhielt. Daher wurde aus diesem Grund versucht, möglichst viele Bauteile und ihre Verkabelungen an ihren vorgesehenen Pins anzulöten. So auch der BBB selbst, der mit seinen 20 genutzten Pins die mit Abstand größte Angriffsfläche für Auflockerungen bot.

Für einen hohen Prozentsatz der Ausfälle konnte der Encoder als Problemfall erkannt werden. Trotz der behobenen anteiligen Systemausfälle konnten weitere Systemausfälle seitens des MD49 nicht behoben werden. Bei dem MD49 Encoder kam es zu ca. 10% der Testfahrten zu weiteren unerklärlichen Systemausfällen. Der Grund für diese Ausfälle konnten dabei leider nur vermutet werden, da bei Ausfällen des Encoders keine Error Meldung zu finden war. Die auf dem MD49 Encoder verbaute „Error LED“³⁵, waren daher bei Systemausfällen nicht leuchtend zu erkennen. Alleinig die „Comms LED“³⁶, die für die Kommunikation Anzeige zuständig war, deaktivierte sich bei den genannten ausfällen.

³⁵ Auf der MD49 Platine verbauten LED die rot aufleuchtet, sobald ein Fehler bei der Übertragung auftritt.

³⁶ Auf der MD49 Platine verbauten LED die gelb aufleuchtet, wenn eine Datenübertragung stattfindet.

4.3 Gegenüberstellung beider Regelungen

In dieser Arbeit wurden zwei Regelungen untersucht, die das Problem der Balancierung unterschiedlich versucht haben zu lösen. Der signifikante Unterschied beider Regelungen ist die Art und Weise, wie sich die Lenkerposition zum Lagesensor verhält.

In der ersten Regelung wurde versucht durch das Koppeln der Geschwindigkeit an einer Mathematischen Funktion, die Balancierung zu ermöglichen. Dabei wurde das Anfahren des Lenkmotors alleinig durch das Erreichen der Encodergrenze oder durch das erneute Aufrechtstellen des Fahrrades abgebrochen. Dieses Verhalten ist in den Aufnahmen gut zu erkennen (siehe *Bachelorarbeit/Aufnahmen/Labor Aufnahmen (Regeler)/1.Regeler*).

Die zweite Regelung allerdings versuchte das Balancierproblem anderes zu lösen. Hierbei wurde versucht die verschiedenen Lagesensorpositionen mit bestimmten Lenkpositionen zu koppeln, sodass jede Schräglage des Fahrrades, mit einer bestimmten Einlenkposition einzuordnen ist. Dadurch bewegt sich der Lenker, parallel zu der Schräglage des Fahrrades. Dabei ist auch dieses Verhalten in den Aufnahmen zu erkennen (siehe *Bachelorarbeit/Aufnahmen/Labor Aufnahmen (Regeler)/2.Regeler*).

Die erste Regelung nutzte als Funktion den Tangens-hyperbolicus³⁷, der durch seine zu Beginn starke Steigung und der damit einhergehenden starken Beschleunigung des Lenkmotors, die Schräglage des Fahrrades schnell auskurieren sollte. Dabei würde sich der Tangens-hyperbolicus anhand eines im Bruch befindlichen Faktors gestaucht oder gestreckt werden. Die Stauchung oder Streckung der Funktion würde die Beschleunigung anhand der Entfernung zum Nullpunkt anpassen. Hierbei war der Faktor davon abhängig wie lange sich das Fahrrad in der davor definierten habitablen Zone³⁸ befände. Diese wurde zu Beginn in einem Wertebereich von -0.002 bis 0.002 Rollwinkel Einheiten festgelegt. Sobald sich das Fahrrad in dieser Zone befände, würde eine Stoppuhr gestartet werden, die die Zeit auf die Nanosekunde genau misst und diesen gemessenen Zeitwert als Faktor für die Stauchung oder Streckung der Funktion übergibt. Der Lenker wird dann anhand dieser Funktion beschleunigt, bis der Lenker die Encodergrenze von 200 Ticks erreicht hat oder sich der IMU wieder in der Habitablen Zone befindet. Im zweiten Fall starte die Prozedur von neu und es kommt zu einer weiteren Zeitmessung.

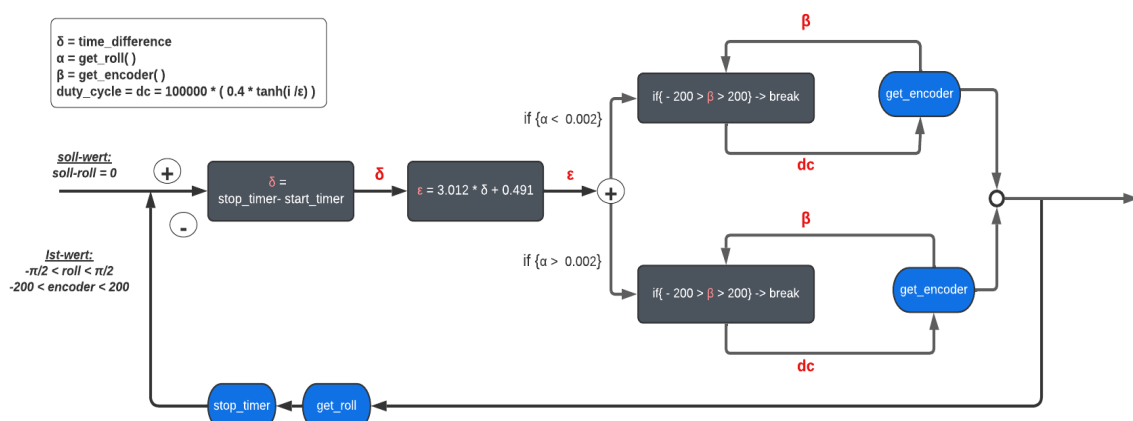


Abbildung 22: Erster Regeler

³⁷ Eine Hyperbelfunktion zur Erstellung einer Geschwindigkeitsfunktion.

³⁸ Eine im übertragenen Sinne Sichere Zone, in der Änderungen des Zustandes mehr nicht nötig sind.

In der *Abbildung 22* wird der Ablauf des Reglers dargestellt. Die Zeit δ bestimmt den Faktor, der für den Streckungs/Stauchungs-Faktor zuständig ist. Sobald dieser Faktor bestimmt ist, kommt es zur Abzweigung des Reglers, welche durch die jeweilige Kipprichtung der IMU begründet ist. Nach dem ein Weg durchlaufen wurde, gelangt man in einer innere Regelschleife, welche so lange durchlaufen wird, bis die Encodergrenze erreicht oder die IMU in die Habitable Zone gelangt. In jedem der inneren Regeldurchläufe kommt es dann zur Abfrage des aktuellen Encoderwertes, welches eine Abfragelatenz von ca. 4ms besitzt (siehe Kapitel 4.2 „vor Optimierung“). Dadurch kann es zu Ungenauigkeiten des Erreichens der Grenze kommen, was durch die in *Abbildung 24* zusehenden Funktion „encoder_liberation_L/R“, korrigiert wird. Die „dc“ Funktion die der „get_encoder“ als Variable übergeben wird, ist dabei von der Funktion

$$dc(i) = 100000 * (0.4 * \tanh(i/\varepsilon)) \text{ abhängig.}$$

Dabei ist die eingetragene 0.4, als Vorfaktor des Tangens- hyperbolcus zu verstehen, der die obere Grenze der maximalen Leistung des Motor widerspiegelt. Dadurch ist ein Maximaler Tastgrad von nunmehr 40% möglich, der durch den Tangens-hyperbolcus durchlaufen wird.



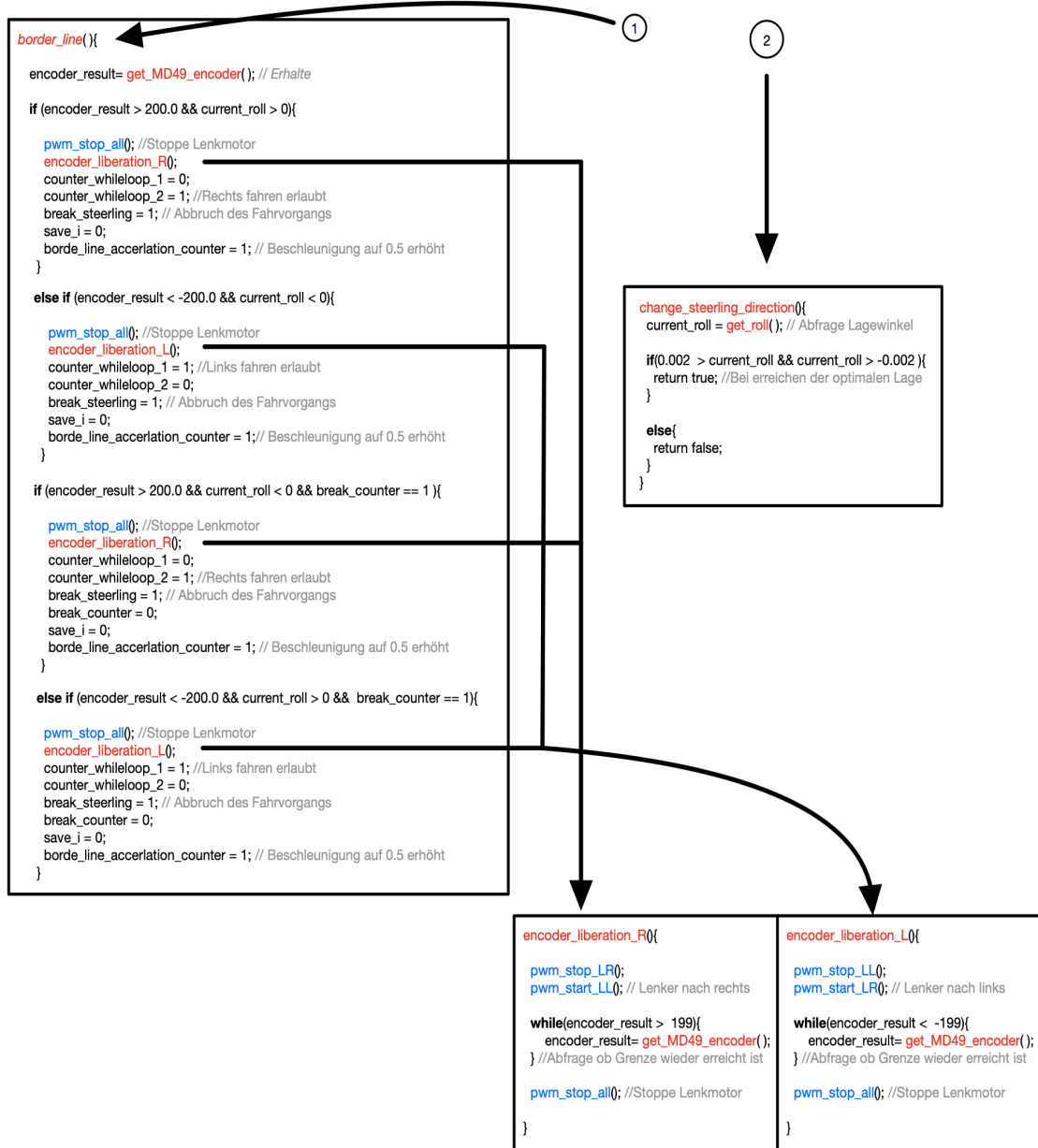


Abbildung 24: Pseudocode der ersten Regelung Teil 2

In der zweiten Regelung wird versucht mittels eines weiteren Ansatzes die Balancierung des Fahrrades zu ermöglichen. Hierbei orientierte man sich deutlich stärker an der Physikalischen Fahrdynamik des Fahrrades und versuchte diese nachzuahmen. Es ist zu erkennen, dass die Fahrdynamik des Fahrrades den Lenker hin zur Kipprichtung bewegt, wodurch sich das Fahrrad wieder aufrichtet. Dieses Verfahren wiederholt sich dann auf der entgegengesetzten Seite Spiegelverkehrt wieder, da sich das Fahrrad zu keiner Zeit wieder auf der optimalen Schräglage zurückrichten kann. Dadurch ist ein ständiges Pendeln³⁹ des Fahrradlenkers zuerkennen, was dazu führt, dass das Fahrrad in keinem Zeitpunkt den Lenker Unbeweglichen lassen würde.

Um ein solches Einlenken zu realisieren, wurde durch Berechnungen eines Sollwertes für den Encoder hergenommen, der als Maßstab für den Lenker zu gelten hat, das Verhalten der Fahrdynamik bestmöglich nachzuahmen. Der Sollwert ist daher hauptsächlich auf die

³⁹ Ein hin- und herbewegen der Lenkerposition sowie dem Encoder Wert im Graphen.

Schräglage des Fahrrades fokussiert und übergibt den Rollwinkel mit einer Verstärkung, als anzufahrender Encoderwert weiter.

Im ersten P-Regler wird zu dem abgefragten Rollwinkel ein Wert multipliziert der zu Beginn des Codes als Parameter übergeben wurde. Wie die Variable `Rf_2` vermuten lässt, gibt es für diese Regelung eine zweite Variante, den Sollwert zu bestimmen. Dieser wird mittels dazunehmen der aktuellen Geschwindigkeit realisiert. Hierbei ist der Vorteil, dass sich der Faktor, mit dem der Rollwinkel multipliziert wird, dynamisch mit der Geschwindigkeit verändert. Dadurch können höhere Geschwindigkeiten dazu führen, dass sich der Lenker deutlich Steifer bewegen lässt, da ein starkes Einlenken dabei kontraproduktiv ist. Die Erkenntnisse zu diesem Verfahren, sowie das Bestimmen der Funktion wird in Kapitel 5.4 erläutert.

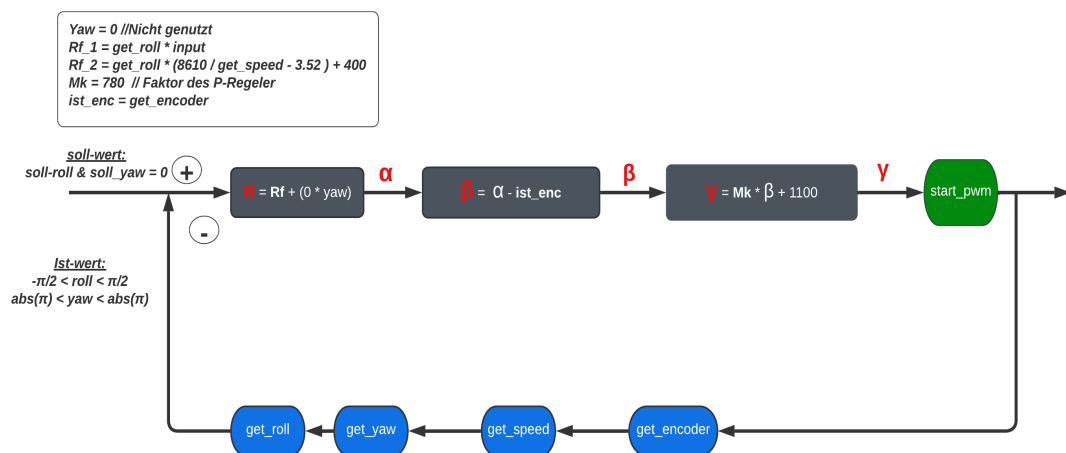


Abbildung 25: Zweiter funktionierender Regler

Der dabei entstehende Soll-Wert α wird dann zu Ermittlung der Regeldifferenz übergeben. Hierbei kann dann, Auskunft darüber gegeben werden, wie weit der Lenker von seiner Zielposition entfernt ist und wie schnell er dabei angefahren werden soll. Die Regeldifferenz wird dann im Anschluss zu einem Tastgrad umgerechnet. Dabei entschied man sich beim Einstellen des Reglers für eine Addition von 11.000, da bei einem Tastgrad von 11.100 der Motor anfangen würde sich zu drehen. Im Anschluss wird der Wert γ in das PWM Verzeichnis des BBB eingeschrieben, welches dann ein PWM Signal entstehen lässt, der an den BTS7690 weitergegeben wird und den EMG49 in Bewegung setzt.

Ein passender Nebeneffekt dieser Regelung ist es, dass eine Dämpfung für den Lenkmotor einbaut ist. Da es sich bei der Transportzeit um eine Laufzeit von ca. 2ms handelt (siehe Kapitel 4.2 „nach Optimierung“), kann es bis zu acht Regeldurchläufen kommen, bis der Soll-Encoder erreicht wurde. Bei diesen acht Regeldurchläufen kommt es zur Aktualisierung der Regeldifferenz, wodurch die Geschwindigkeit des Motors gleichzeitig reduziert wird.

Darüber hinaus sind in dieser Regelung, anders als in der ersten Regelung, keine Encodergrenzen mit einzubeziehen. Dadurch, dass die maximale Schräglage durch die Stützräder beschränkt sind, sind diese auch zur selben Zeit als Encoder Grenzen zu verstehen.

Die dadurch vereinfachte Design-Sprache des Reglers ist dadurch auch in der Komplexität im Vergleich zum ersten Regler gut zu erkennen (siehe *Abbildung 26*).

Das Designen und Testen der zweiten Regelung nahm in dieser Arbeit den mit Abstand größten Teil ein. Beim Arbeiten mit dieser Regelung kamen einige Ideen zusammen, die zum größten Teil umgesetzt worden sind. Diese Ideen entstanden durch das Erkennen von Schwachstellen, welche versucht wurden, zu beheben. Ein Beispiel dafür ist die Geschwindigkeitsmessung. Diese entstand durch den Versuch, die Messungen in einem möglichst konstanten Umfeld durchzuführen. Da sich bereits erkennen ließ, dass die unebene Teststrecke die Messwerte verfälschte, musste versucht werden, die Steigung und die damit einhergehenden Verfälschungen der Geschwindigkeit auszuhebeln. Dadurch entstand der Gedanke, die Geschwindigkeit zu messen und diese durch einen Regler⁴⁰ möglichst konstant zu halten.

⁴⁰ Regler im Code „P_Regler.c“ zu finden

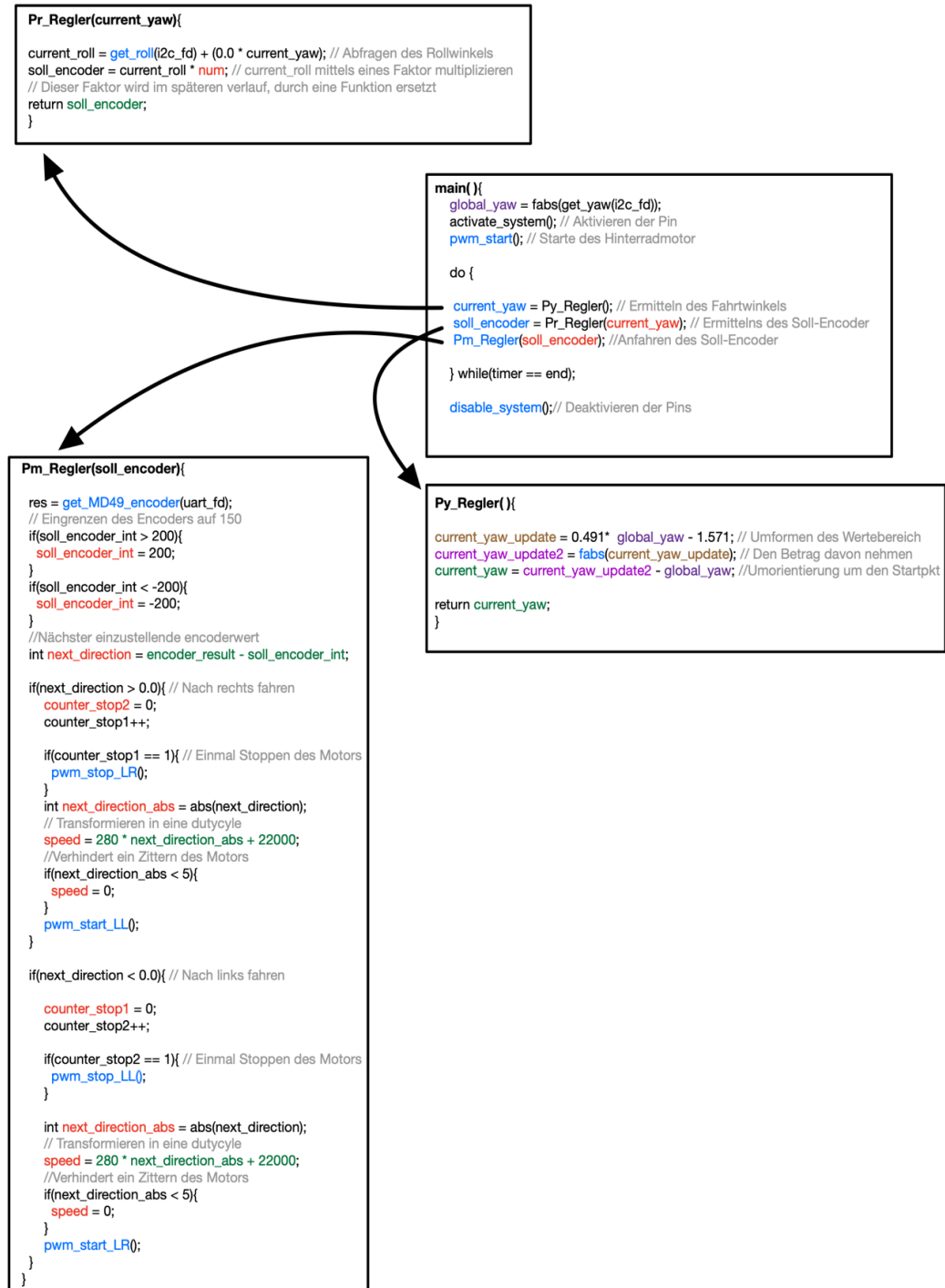


Abbildung 26: Pseudocode zur zweiten Regelung

Zum Ende des Codes hin, wird die „disable_system“⁴¹ Funktion ausgeführt, die alle wichtigen Systeme runterfährt. Dabei werden die PWM, PRU und GPIO Schnittstellen deaktiviert und geschlossen, sowie die ursprüngliche Lenkerposition angefahren. Die im Array befindlichen Rohdaten werden in einer csv-Datei abgespeichert und zum Schluss geschlossen und im selben Verzeichnis verschoben.

⁴¹ Siehe in Abbildung 26 in der Funktion „Py_Regler“.

5 Evaluation

In diesem Kapitel wird versucht, dem Leser zu vermitteln, wie die aufgeführten Diagramme zu verstehen sind und wie diese zu Stande kamen. Zu Beginn werden Bewertungskriterien gesetzt, welche dem Leser ermöglichen sollen, die Bewertungen der jeweiligen Regler besser nachvollziehen zu können.

5.1 Bewertungskriterien

Für die Bestimmung einer passenden Regelung sowie den dazugehörigen Parametern, sind bestimmte Bewertungskriterien festzuhalten. Diese bestimmen, wie gut ausgeführte Testfahrten zu bewerten sind und ermöglichen es, Rückschlüsse auf die darauf verwendeten Parameter zu führen. Dabei werden die Kriterien in verschiedenen Stufen eingeteilt, die ihre Wichtigkeit durch ihre Platzierung widerspiegeln sollen. Bei den gesetzten Kriterien handelt es sich um folgende:

1. Ein möglichst geringes Pendeln des Lenkers:

Dieses Kriterium wurde gewählt, da sich ein geringes Pendeln des Lenkers, auf die stabile Fahrweise des Fahrrades auswirken lässt. Ein solches Pendeln kann bei zu großem Regelparameter zu einem unkontrollierten ausschlagen des Lenkers führen. Gleichzeitig kann es bei zu geringer Parameter Wahl dazu kommen, dass sich der Lenker nicht ausreichend einlenken lässt. Man sollte sich daher in einem Encoder Bereich von -50 bis 50 bewegen, welches noch als ein Pendeln im Rahmen einzuordnen ist. Daher ist ein geringes Pendeln das wichtigste Kriterium für eine gute Balancierregelung.

2. Eine möglichst lange Fahrdauer:

Eine möglichst lange Fahrdauer ohne einen Zwischenfall ist für eine Bewertung des Reglers ausschlaggebend. Dabei hängt dieses Kriterium zu einem gewissen Teil vom ersten Kriterium ab. Hält sich das Pendel des Encoders im Rahmen, so kann eine lange Fahrdauer sichergestellt werden. Trotz allem kann es bei einem hohen Pendelgrad zu einem eher schlechten Fahrverhalten kommen, wobei das Fahrrad die verbauten Stützräder in der gesamten Fahrdauer nicht berührt.

3. Eine möglichst gerade Fahrweise:

Das letzte Kriterium ist für die Arbeit noch nicht von großer Bedeutung gewesen, wird aber trotzdem in die Arbeit mit eingenommen, da sich eine gerade Fahrweise für zukünftige Arbeiten als sehr wichtig herausstellen wird. Eine gerade Fahrweise, lässt sich daraus erkennen, dass sich das Pendeln, gleichmäßig um den Nullpunkt bewegt. Dieses Kriterium hängt genau wie das zweite Kriterium vom ersten Kriterium ab.

5.2 Messdaten

Um die Testfahrten bestmöglich festzuhalten, wurden in einer csv-Datei⁴² alle für die Regelung wichtigen Parameter mitverfolgt und abgespeichert. Diese Parameter konnten dann im Nachhinein eingesehen und mittels eines Diagramms verbildlicht dargestellt werden. Bei den genutzten Parametern handelte es sich um folgende Werte:

1. **Aktuelle Zeit:** Durch die im Code vermerkten Zeitstempel konnten durch bestimmte Zeitabstände nachverfolgt werden, wie stark sich die Parameter verändern und ob sich ein Muster des Regelverhalten erkennen ließ.
2. **Zeitdifferenzen:** Durch die Differenz der aktuellen und vergangenen Zeitstempel berechneten Zeitdifferenzen ermöglichte man eine genaue Ermittlung der Transportzeit. Die dabei entstandenen Zeitdifferenzen werden im Kapitel 5.4 genauer behandelt und erklärt.
3. **Ist - Encoder:** Der Ist-Encoder gibt die aktuelle Position des Lenkers an. Hierbei wird ein Drehen des Lenkers um 360° als ein Maß von 980 Möglichen Ticks⁴³ gewertet. Dabei kann der Encoder bei Bedarf zurückgesetzt werden, um verschiedene Positionen als Nullpunkt zu werten.
4. **Soll - Encoder:** Der Soll-Encoder gibt den anzufahrenden Ist-Encoder Tick an und wird als Maß für die Lenkerposition einer Balancierfähigkeit gewertet. Dieser wird vom Rollwinkel des Lagesensors vorgegeben und dient als Übersetzung der aktuellen Lage. Man versucht hierbei die Theorie der Fahrdynamik mittels Einlenkens in Kipprichtung möglichst gut umzusetzen. In Diagrammen wird dann dieses Kippverhalten durch ein Pendeln des Soll-Encoders veranschaulicht.
5. **Yaw - Winkel:** Der Yaw-Winkel ist ein Maß für die Fahrtrichtung des Fahrrades. Hierbei wird mittels eines Wertebereiches von $-\pi$ bis π die möglichen Fahrtrichtungen von 360° veranschaulicht. Durch das Hinzunehmen des Betrages auf den Wertebereich werden die möglichen Fahrtrichtungen gespiegelt und erhalten eine kontinuierliche Positionswertung.
6. **Roll - Winkel:** Der Roll-Winkel gibt den aktuellen Kippwinkel des Fahrrades an. Dabei wird eine exakt gerade Fahrweise mit einer 0 bewertet und ein Kippen in die jeweilige Richtung durch einen Wertebereich von $-\frac{\pi}{2}$ bis $\frac{\pi}{2}$ gewertet. Dieser Parameter ist dann ausschlaggebend für die Stärke des Einlenkens und ist nur durch einen Vorfaktor als anzufahrender Encoder Wert zu behandeln.
7. **Periode:** Die Periode ist die durch die PRU errechnete Frequenz, die angibt, wie schnell sich das Fahrrad bzw. das Hinterrad dreht. Diese wird jede Sekunde aktualisiert. Eine Verzögerung der Übertragung ist daher nicht zu vermeiden.

⁴² Ein Dateiformat zum Speichern von gemessenen Rohdaten.

⁴³ Ein Encoder Tick ist eine vom Encoder mögliche einzunehmende Position. Für eine Rotation von 360° , können 980 verschiedene Encoder Ticks eingenommen werden.

- 8. Bike-Speed:** Die Bike-Speed ist ein durch die Periode errechneter Wert, der die aktuelle Geschwindigkeit des Fahrrades angibt. Diese wird in der Einheit km/h angegeben. In den kommenden Testfällen bewegte man sich im Bereich von 6 bis 10km/h. Dieser Geschwindigkeitsbereich wurde gewählt, da vermutet wurde, dass sich die Balancierfähigkeit ab einer Geschwindigkeit von ca. 4km/h einsetzen würde [7].

5.3 Bewertung des ersten Reglers

Dieser Regler wurde zu Beginn der Arbeit entwickelt. Die Idee sowie die Umsetzung waren recht kompliziert, wodurch schon vor dem Starten der Testfahrten vermutet werden konnte, dass sich ein Einstellen der Parameter als sehr schwierig erweisen würde. Eines der Gründe dafür waren die Zeitmessungen des IMU in der habitablen Zone. Hierbei war die Dauer des Eintretens sowie die Größe der Habitable Zone nur sehr schwer zu ermitteln. Bei ersten Probefahrten (ohne Antrieb) im Labor war sehr gut ersichtlich, dass sich die Regelung sehr stabil bei kleineren Neigungen verhält (siehe Bachelorarbeit/Aufnahmen/Labor Aufnahmen (Regeler)). Bei Eintreten in die Habitable Zone verhielt sich der Lenker ruhig und konnte bei langem Hochzählen der Zeit durch die Stoppuhr, mittels gestrecktem Tanges-hyperbolicus, die Geschwindigkeit langsam erhöhen und das Kippverhalten des Fahrrads gut ausgleichen. Nur konnten sich diese Erfahrungen nicht auf der Teststecke bewahrheiten.

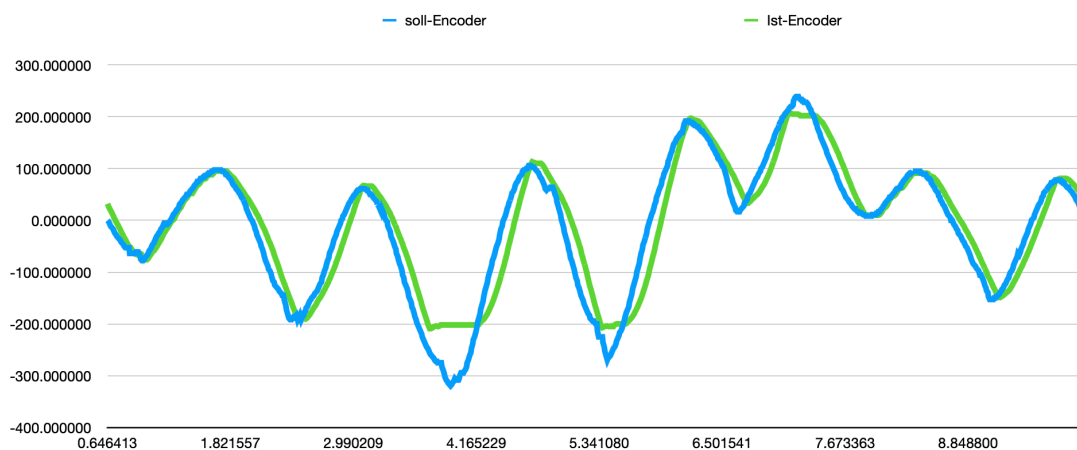


Abbildung 27: Testfahrt mit Erster Regelung

In Abbildung 25 ist gut zu erkennen, dass sich die Regelung für den Anwendungszweck nicht gut einsetzen ließ. Da sich auch dabei kein Muster für die Parameterwahl erkennen lassen hat, wurden weitere Maßnahmen zur optimieren der Regelung fallen gelassen. Darüber hinaus erkennt man, dass sich das Pendeln nicht kontrollieren ließ, weshalb es zum mehrmaligen Erreichen der Encodergrenzen gekommen ist. Dieses ist daran zu erkennen, dass der Ist-Wert dem Soll-Wert in den Sekunden 4, 5.5 sowie 7.6 nicht mehr folgte. Dieses Verhalten führte dazu, dass die Stützräder in so gut wie jedem der Einlenkverfahren erreicht wurden und sich daher die Regelung als nicht nützlich erwies.

5.4 Bewertung des zweiten Reglers

In diesem Kapitel werden die durch die Testfahrten ermittelten Parameter anhand der hier zu erkennenden Diagramme verteidigt.

Bei der Bewertung des zweiten Reglers kam es zu fünf Testfahrten. Drei dieser Testfahrten waren dabei am zielführendsten. Sie unterteilten sich in:

1. In der ersten der drei Testfahrten wurde zu Beginn beobachtet wie sich der Regler verhält und ob dieser sich für unsere Zwecke einsetzen lässt. Dabei erkannte man schnell, dass sich diese Regelung im Gegensatz zur ersten Regelung deutlich sicherer verhielt. Die Aufnahmen (siehe /Aufnahmen/Testfahrten/Erste Testfahrt) verdeutlicht, wie sich das Fahrrad auf bestimmte Neigungen verhielt. Dadurch wurde ein Umkippen verhindert. Man sah sehr gut, dass sich das Eingreifen der Regelung sehr schnell aktivierte und das Pendeln des Lenkers für einen Außenstehenden kaum erkennbar war.
2. In der zweiten Fahrt wurde der Regler ohne Hinterradantrieb betrieben. Es musste geprüft werden, wie das Fahrrad auf starke Geschwindigkeitsänderungen reagiert und diese Änderungen ausbalanciert. Man erkannte schnell, dass sich der Regler für einen solchen Einsatzzweck noch nicht eignete, was in den Aufnahmen (siehe /Aufnahmen/Testfahrten/zweite Testfahrt) zu sehen ist. Man entschloss sich, die Parameter des Pr_Reglers, variable zu halten, sodass sich dieser der Geschwindigkeit anpassen würde. Daher wurde sich als Ziel für darauffolgende Testfahrt, die Messung der Fahrgeschwindigkeit gesetzt, um die Optimalen Parameterwerte der einzelnen Geschwindigkeiten (6km/h bis 10km/h) zu ermitteln.
3. In der letzten Fahrt wurden für Geschwindigkeiten von 6km/h bis 10km/h geprüft, welche Parameter sich am besten einsetzen lassen. Um diese Werte ermitteln zu können, wurde für jede der einzelnen Geschwindigkeiten ermittelt, welche Parameterwerte eine möglichst gute Fahrweise hervorbringen. Mit den erhaltenen fünf Parametern war es nun möglich, diese in einem Koordinatensystem einzufügen und eine Funktion auf diese abzubilden.

Im Folgenden wird nun versucht die dabei erhaltenen Parameterwerte anhand der erzeugten Diagramme zu erklären und zu beschreiben. Diese Auswahl an Diagrammen sind eine Zusammenstellung aus verschiedenen Testfahrten, die als erfolgreich angesehen wurden und dessen Parameterwahl sich als ausreichend für die Balancierung bei einer solchen Geschwindigkeit herausstellte. Sie erfüllen alle der drei Bewertungskriterien und bilden damit eine Zusammenstellung der verwendeten Parameter bei gewünschter Geschwindigkeit.

Es wurde bei den Testfahrten mit einer Geschwindigkeit von 10km/h begonnen, da diese als einfacher zu regeln vermutet wurde. Dies stellte sich im Nachhinein als Trugschluss heraus, da man feststellte, dass sich die Regelung bei einer Geschwindigkeit von 6km/h am besten verhielt.

Für diese Fahrt kam es zu einem Parameterwert von **1750**. Man erkennt bei der *Abbildung 26* sehr schnell, dass sich der P-Regler für die Geschwindigkeitskonstanzierung als nicht zuverlässig erwies. Ein weiterer Grund für das nicht Erreichen der gewünschten 10km/h, könnte eine Verzögerung von der Übertragung der Geschwindigkeit sein. Es ist gut zu erkennen, dass nach ca. 500ms der Geschwindigkeitsmesser aktiviert wird, was darauf vermuten lässt, dass es sich dabei um eine Verzögerung der Übertragung handeln muss. Man sieht, dass der Lenker nur sehr langsam pendelt und, dass das Fahrrad sechs größere Kurven fährt.

Des Weiteren erkennt man eine Tendenz zu den positiven Encodern Werten, was auf eine Rechtsfahrt schließen lässt. Da es sich hierbei leider um eine eher kürzere Fahrt von nur 5 Sekunden hält, kann man nur darüber spekulieren was im weiteren Verlauf geschehen wäre. Man erkennt zumindest zum Ende hin, eine Verkleinerung der Amplitude, was darauf schließt, dass sich das Pendeln wieder verkleinern lässt.

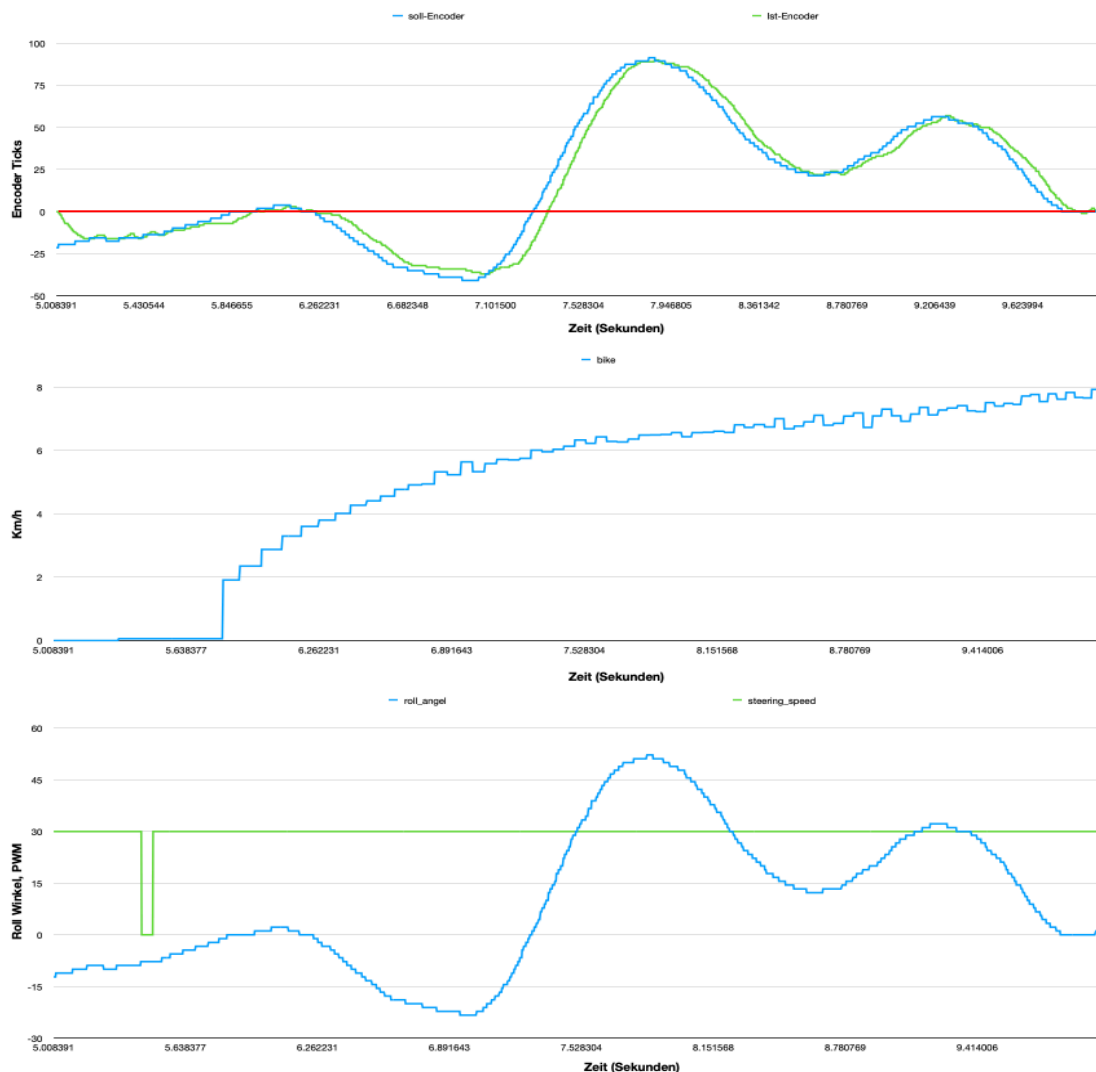


Abbildung 28: Diagramm bei 10km/h und 1750 als Inputparameter

In dieser Testfahrt hat sich der Parameterwert **2000**, am ehesten für eine Geschwindigkeit von 9km/h angeboten. Es ist zu erkennen, dass sich trotz längerer Fahrzeit die gewünschte Geschwindigkeit von 9km/h nicht erreichen ließ. Stattdessen wurde mit einer Geschwindigkeit von ca. 8km/h gefahren. Leider ist zu erkennen, dass sich der Encoderwert zu Beginn weit vom Soll-Wert befand. Diese Differenz ließ das Fahrrad dann stark auskurieren, was zu einer großen Amplitude zu Beginn führte. Daraufhin hat sich das Fahrrad ab der Sekunde 2.4, sehr gut auskuriiert, was ihn dann für ganze 3.6 Sekunden kaum oszillieren ließ. Im Anschluss kam es dazu, dass sich der Soll-Wert nicht mehr einfangen ließ und der Lenker immer weiter auspendelte. Dieses Oszillieren ist in den Aufnahmen, zum Schluss hin sehr gut zu erkennen (siehe /Bachelorarbeit/Aufnahmen/Testfahrten/Dritte Fahrt/ 9km-h bei 2000.mov). Was dabei auch sehr gut zu erkennen ist, ist dass sich das Ausschlagen des Lenkers zu Beginn nicht wie erwartet um die Null bewegte, sondern sich hierbei bei ca. 20 Encoder Ticks bewegte und im weiteren Verlauf sich hin zur 0 bewegte. Das lässt vermuten, dass ein Pendeln um den Nullpunkt, vom Fahrrad angestrebt wird.

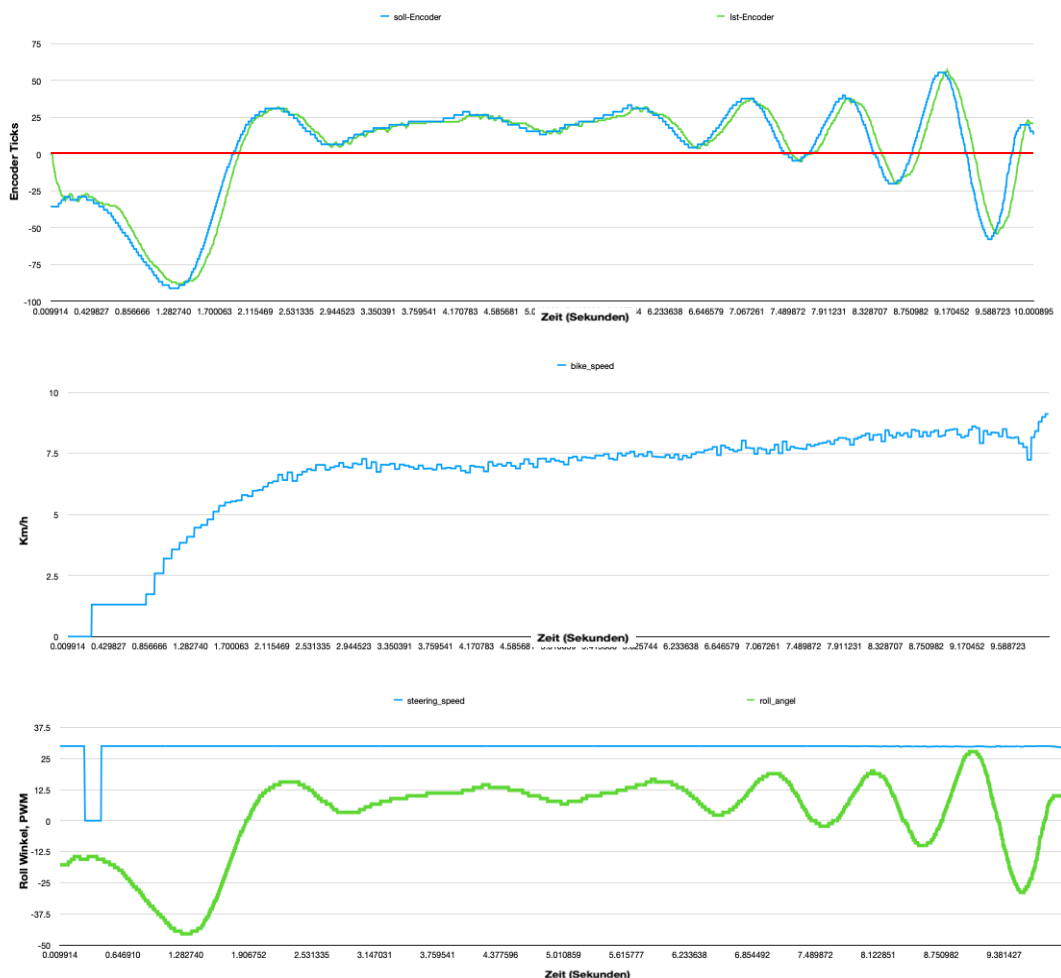


Abbildung 29: Diagramm bei 9km/h und 2000 als Inputparameter

Am PWM Wert des Lenkers ist eine fast durchgängige hohe Last des Tastgrades zu sehen. Man erkennt, dass sich dieser nach ca. 0.3 Sekunden auf den Nullwert zurückbewegt, da sich der Ist-Encoder zu diesem Zeitpunkt, an seiner Piek Position befindet und kurz stillsteht. Das plötzliche Abfallen des Tastgrades ist durch die gesetzte Untergrenze zu erklären, die das direkte Anfahren des Motors bei kleinstem Kippen des Sensors sicherstellt.

Anhand der *Abbildung 29* wird die Testfahrt mit einer Geschwindigkeit von 8km/h dargestellt. Der nun hier eingetragene Parameterwert beträgt **2125**. Man erkennt sehr deutlich im Verhältnis zu den vergangenen Messungen, dass sich das Pendeln des Lenkers deutlich verringerte. Das erste stärkere Ausschlagen des Encoders ist auf die anfängliche falsche Positionierung zurückzuführen, wodurch sich die zweite Schwingung auf einen etwas Abgeschwächten Encoder Wert zurückfallen lässt. Ab ca. 2.7 Sekunden fängt das Fahrrad an, seine Balance zu erhalten und beginnt sehr gleichmäßig, um den Nullpunkt mit einem Peak von etwa 20 Encoder Ticks zu pendeln. Ein solcher Effekt deutet auf eine sehr gerade Fahrweise hin, die durch eine sehr geringe, aber dafür gleichmäßige Kurvenfahrt in beide Richtungen hindeutet. An der Geschwindigkeitsmessung ist hierbei auch zu erkennen, dass die angestrebten 8km/h um ca. 0.5km/h verfehlt wurden und somit die Zielgeschwindigkeit zum wiederholten Mal nicht erreicht werden konnte.



Abbildung 30: Diagramm bei 8km/h und 2125 als Inputparameter

Am Verhalten des Tastgrades des PWM Signals des Lenkers erkennt man sehr gut, dass es zu zwei Abfällen des Tastgrades gekommen ist. Zum einem beim ersten Piek, sowie beim Richtungswechsel des Lenkers nach ca. 1.1 Sekunden.

Was nun bei der Fahrt mit 7km/h zu erkennen ist, bestätigt die vorherige Vermutung. Bei einem Start von ca. 55 anzufahrenden Encoder Ticks wird nach einem starken Ausschlagen der Lenker, die Amplitude deutlich kleiner. Der Soll- sowie Ist-Encoder bewegen sich beide zum Nullpunkt hin, erreichen aber dann ein Oszillieren um die 22.5 Encoder Ticks. Man kann hier sehr gut erkennen, dass dieser Effekt ab der sechsten Sekunde eintritt. Zusätzlich sieht man, dass sich die drei Schwingungen zum Schluss hin leicht verkleinern lassen, sodass der letzte Pegel eine Größe von nun mehr ca. 11 Encoder Ticks aufzuweisen hat. Wie auch in den vorherigen Fahrten wurden hier die angestrebten 7km/h nicht komplett erreicht. Man bewegte sich hier bei einer Geschwindigkeit von etwa. 6.3km/h.

In dieser Aufnahme ist wiederum zu erkennen, dass sich die Geschwindigkeitsaufnahme erst ab einer Zeit von etwa 1 Sekunde eintreten.

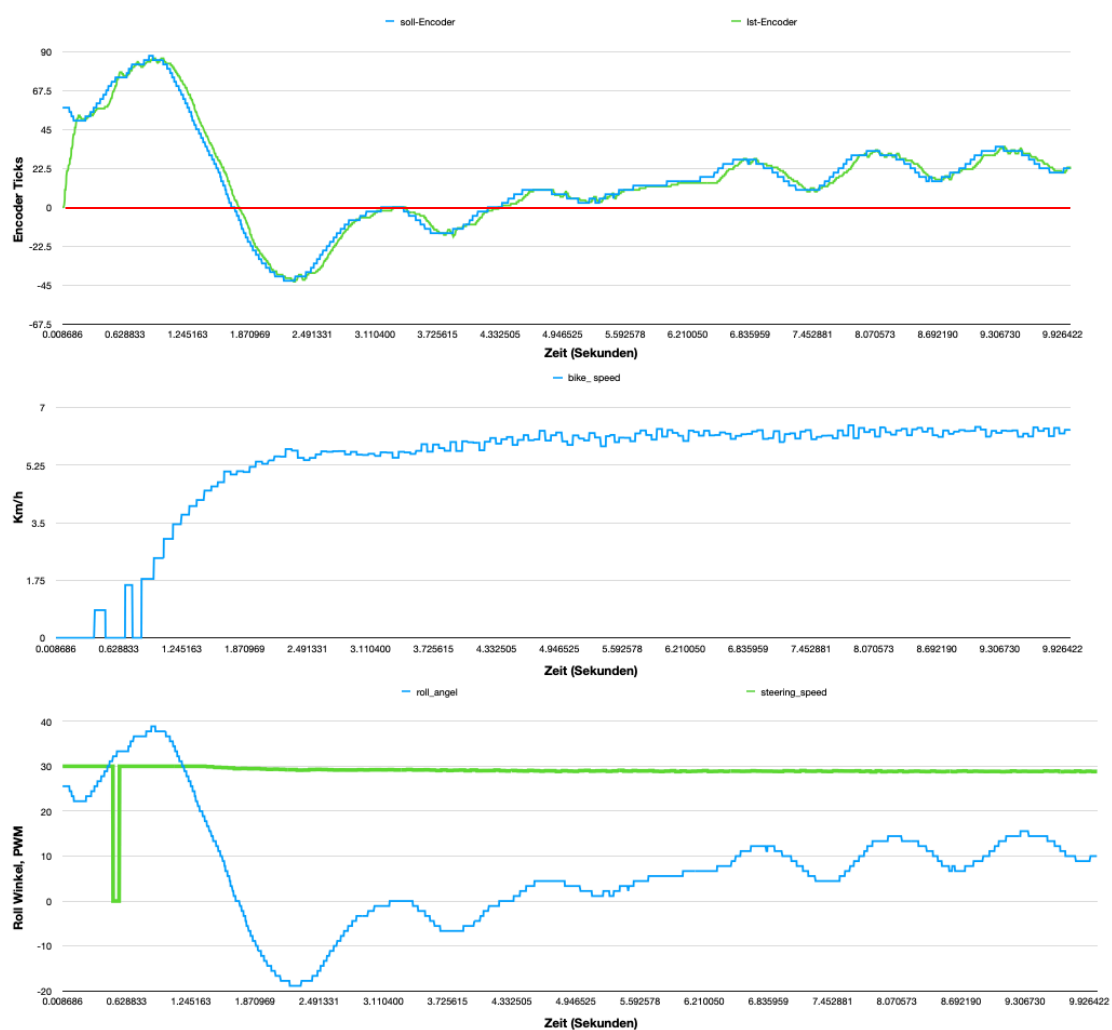


Abbildung 31: Diagramm bei 7km/h und 2250 als Inputparameter

Ähnlich zu den vorherigen Aufnahmen sieht man hierbei wieder, dass sich zu Beginn der Fahrt, das PWM Signal für wenige Millisekunden ausschaltet, da sich der Lenker zu diesem Zeitpunkt nicht bewegen musste.

Bei einer Fahrt mit 6km/h ist ein wiederholter falscher Startwinkel zu erkennen, der dazu führte, dass sich der Soll-Encoderwert bei ca. 40 anzufahrenden Encoderticks befand. Dies führte zu einem nicht erklärlichen Pegelausschlag von ganzen 95 Encoder Ticks, die dann im Nachhinein durch ein zügiges Einpegeln auskuriert wurden. Was bedauerlicherweise ebenfalls zu erkennen ist, ist die Tatsache, dass sich die Schwingungen nicht um den Nullpunkt bewegen und sich daher um die Rollwinkel von ca. 6.5 aufhielten. Das führt dazu, dass sich das Fahrrad leicht in einer Linkskurve bewegt und sich aus dieser vermutlich auch nicht befreit hätte. Hierfür wäre ein I-Anteil, den man am P-Regler hinzufügt, ratsam gewesen.



Abbildung 32: Diagramm bei 6km/h und 4500 als Inputparamter

Was in *Abbildung 33* zu erkennen ist, ist, dass sich der Peak des Pegels weiter auf nur 7.5 Encoderticks reduziert hat. Darauf hindeutet, dass sich dieser bei weiter verkleinerter Geschwindigkeit immer weiter reduziert.

Die hierbei verwendeten csv-Aufnahmen sind unter dem Verzeichnis (*Bachelorarbeit/Aufnahmen/csv Aufnahmen/**) zu finden.



Abbildung 33: Nahaufnahme der Schwingung

Schon vor Antreten der Testfahrten wurde vermutet, dass es sich bei der Funktion, die der Parameterwahl nahekommen soll, um eine Hyperbel handelt. Bei der gewählten Funktion $f(x) = \frac{8610}{x-3.52} + 400$ ist eine Approximation gegen 400 zu erkennen. Diese Funktion wird dann in den P-Regler, der zum Bestimmen des Soll-Werts zuständig ist, einbaut und dient dann zu dynamischen Soll-Wert Wahl (siehe Abbildung 25).

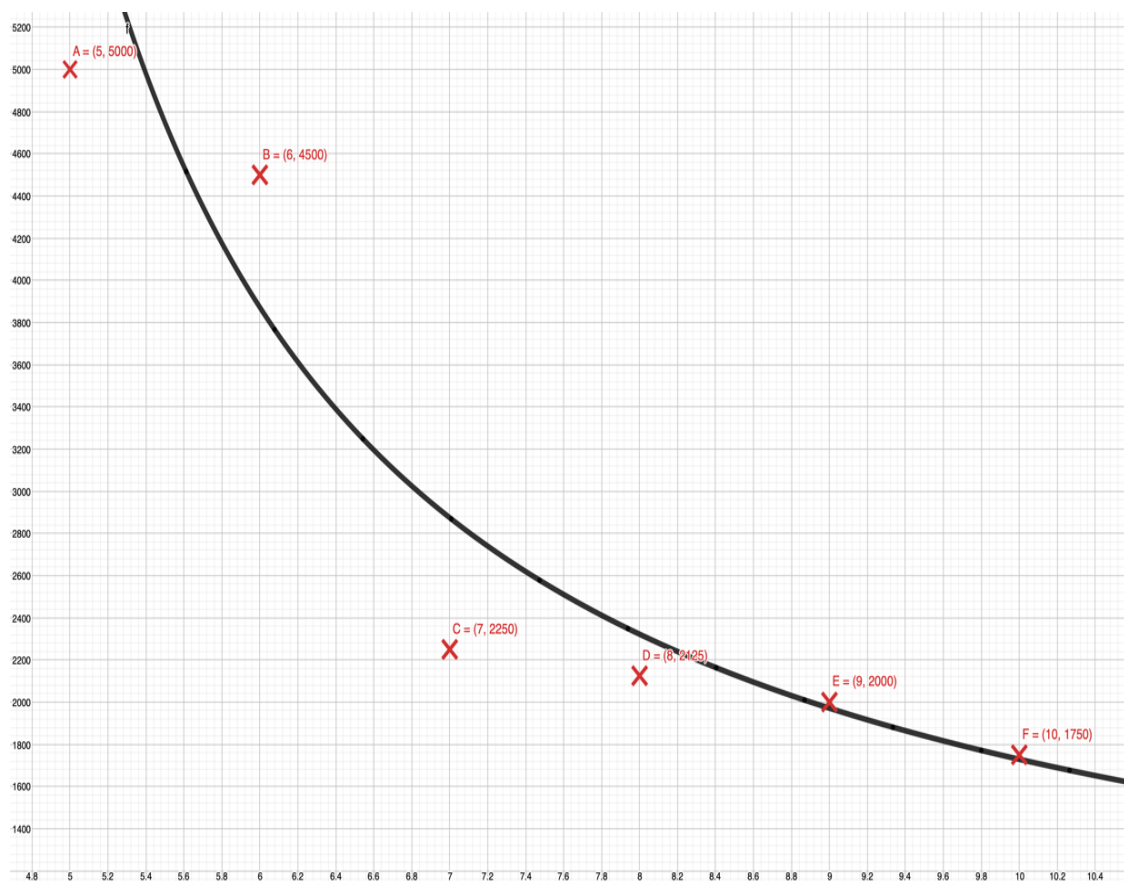


Abbildung 34: Funktion $f(x) = \frac{8610}{x-3.52} + 400$ für Parameterwahl

Es ist nicht mehr nötig, eine bestimmte Geschwindigkeit bei Start des Codes anzugeben. Gleichzeitig ist ein halten einer Konstanten Geschwindigkeit damit auch hinfällig. Das Integrieren dieser Funktion, in den Regelkreis würde darüber hinaus das Fahren auf der Teststreck deutlich vereinfachen.

6 Fazit

Durch diese Arbeit ermöglichte man es, eine Balanceregung für ein Fahrrad zu entwickeln. Trotz längerer Bearbeitungszeit waren diese Monate sehr lehrreich und gestatteten einen Einblick in verschiedene Bereiche der Informatik. Diese Arbeit ermöglichte den zweiten Schritt für eine Vielzahl von weiteren Arbeiten, die in Zukunft zu diesem Thema verfasst werden können.

6.1 Ergebnisse

Anhand der Messungen konnten eine Reihe an Schlussfolgerungen gemacht werden. Zum einen muss gesagt sein, dass sich die Messungen auf einer geraden und ebenen Fläche für erste Schritte deutlich besser angeboten hätte, um daraus Schlüsse für Bedingungen ziehen zu können, jedoch erwies sich die in Kapitel 4.1 erwähnte Strecke als ausreichend. Man erkennt ebenfalls, dass sich das Fahrrad für das Konzept der selbst Balancierung noch in den Kinderschuhen befindet. Es ist bisher auf sehr vielen Ebenen unzuverlässig und ist für Testfahrten noch recht intuitiv aufgebaut, weshalb an ihr noch einiges zu bearbeiten ist (siehe Kapitel 5.4). Ein weiterer Punkt, welcher aufgefallen ist, ist die Unzuverlässigkeit des Geschwindigkeitsmessers. Es war auffällig, dass dieser für Geschwindigkeiten um die 2km/h nicht einsetzbar ist.

Tatsächlich ist es in keiner der Probefahrten gelungen, den IMU Sensor zu Beginn möglichst gerade halten zu können, sodass das Fahrrad ohne starke Schwingungen des Lenkers zu Beginn auskam. Dieser musste sich leider im Laufe der fahren, ständig auskurieren damit dieser um den Nullwert als Soll-Wert oszilliert.

6.2 Ausblick

Um nun einen Ausblick für kommende Arbeiten zu beschreiben, wird sich hierbei möglichst ausführlich gehalten:

1. Eine der größten Schwachstellen der Regelung ist die Verfehlung des exakten Winkelnulldpunktes. Diese Verfehlung führte dazu, dass sich das System immer wieder zu Beginn auskurieren musste, was unnötige verbrauchende Ressourcen und Risiken mit sich brachte. Man müsste bei Testfahrten unabhängig vom verbundenen Notebook erkennen, wann dieser Nullpunkt erreicht ist. Eine mögliche Idee wäre es, durch einen kleinen verbauten Lautsprecher dem Nutzer ein Signal zu übergeben, der diesem vermittelt, sich nahe am Nullpunkt zu befinden. Es wäre wahrscheinlich hilfreich, die Lautstärke des Lautsprechers davon abhängig zu machen, wie weit sich der Neigungsfaktor von dem exakten Nullpunkt entfernt ist. Dadurch würde ein zu Beginn großer Encoder Ausschlag verhindert werden können.
2. Eine weitere Schwachstelle des Systems ist es, dass sich das Fahrrad nur selten zu einer geraden Fahrweise bringen lässt. Schon die kleinsten Fehler im System führten dazu, dass sich das Fahrrad in eine Richtung fahren lassen hat und dabei vom Kurs abkam. Dieser Effekt kann sehr wahrscheinlich durch das Hinzunehmen des Yaw Winkels eliminiert werden. Durch diesen Winkel ist es dem System möglich die Fahrtrichtung mitzuverfolgen, was ihn dazu bringen

könnte bei Abweichungen gegenzusteuern, wodurch ein gerades Fahren gewährleistet werden kann.

3. Ein Wichtiger Punkt ist das Hinzunehmen von weiterer Sensorik. Für spätere Arbeiten wäre es sinnvoll nach dem Lösen des Problems in Punkt 2, sich auf die Eingliederung im Straßenverkehr zu fokussieren. Für ein solches Vorhaben wären daher das Verbauen weiterer Sensoren sehr hilfreich. Man könnte sich mittels Lidar Sensorik⁴⁴ und/oder einer Kamera Sensorik einen Überblick von seiner Umgebung verschaffen und die dadurch erhaltenen Daten dafür nutzen sich z.B. vor Zusammenstürzen und Unfällen zu schützen.

Ein wichtiger Punkt, der sich bei Hinzunahme weiterer Sensoren ergibt, ist die Einhaltung der 5ms Grenze. Diese sollte bei einem gegebenen Puffer von etwa 3ms nicht überschritten werden, um die Reaktionssicherheit des Fahrrades weiterhin gewährleisten zu können. Da eine Objekterkennung durch Hinzunahme eines Umgebungsaufnahmegerät einiges an Ressourcen benötigt, wäre die Hinzunahme eines weiteren Mikrocontroller ratsam. Man könnte dadurch die verbleibenden Ressourcen des BBB einsparen und diese nur noch dafür nutzen eine Kommunikation mit dem weiteren Mikrocontroller aufrechtzuhalten, sowie für einfachere weitere Aufgaben wie in Punkt 1 beschrieben.

Für Aufgaben, die sich im Bereich der künstlichen Intelligenz befinden, wäre die Hinzunahme eines Beaglebone AI, der sich auf diesem Themenfeld spezialisiert hat, sehr hilfreich.

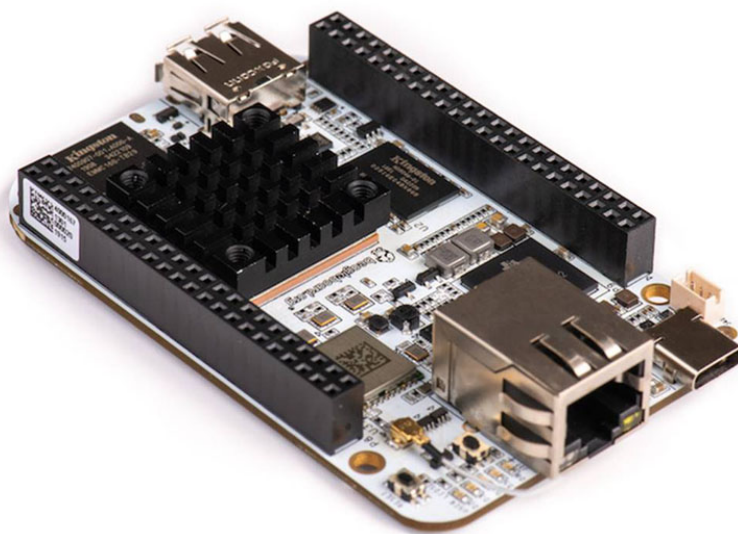


Abbildung 35: Beaglebone AI

⁴⁴ Ein Sensor der die Umgebung abtastet und diese in ein 3D- Model abbildet.

Literaturverzeichnis

- [1] B. f. D. u. Verkehr, „Radverkehr“, 2022.
- [2] P. Zöhl, „Auf zwei Rädern“, Deutsche Welle, 2013.
- [3] B. Brackel, „Marokko baut an der Zukunft der Solarenergie“, Süddeutsche Zeitung, 2016.
- [4] B. Finke, „Wie hart ist das Verbrenner-Verbot der EU?“, Süddeutsche Zeitung, 2022.
- [5] K. Schweighöfer, „Fahrradland Niederlande, warum das Fahrrad so beliebt ist“, Deutschland.de, 2019.
- [6] Google, Regisseur, *Introducing the self-driving bicycle in the Netherlands*. [Film]. Niederlande.2016.
- [7] A. Ranz, How to make a Bicycle (not) fall - implementing a steering control for a self balancing bicycle, 2021.
- [8] S. Viehmann, Happy Birthday, Cruise Control, Süddeutsche Zeitung , 2010.
- [9] J. Morris, „Why Is Tesla’s Full Self-Driving Only Level 2 Autonomous?“, *Forbes*, 2021.
- [10] P.-D. D.-I. S. Schmidt, „Magdeburger wollen autonome Lastenräder“, 2021. [Online]. Available:
https://www.ovgu.de/Presse+_+Medien/Pressemitteilungen/PM+2021/März/PM+13_2021-p-112452.html.
- [11] F. Reek, „Dieses E-Bike braucht keinen Fahrer“, Süddeutsche Zeitung, 2019.
- [12] P. Zhihui, „Github“, 2021. [Online]. Available: <https://github.com/peng-zhihui/XUAN>.
- [13] S. Marc, „280km/h mit dem Fahrrad“, *Faz*, 2020.
- [14] robotshop, „MD49 Datasheet“.
- [15] U. Probst, „Leistungselektronik für Bachelors Grundlagen und praktische Anwendungen“, Hanser, 2020.
- [16] D. Molly, „Exploring BeagleBone: Tools and Techniques for Building with Embedded Linux“.
- [17] P. Balzer, „cbccity“, 2012. [Online]. Available: <https://www.cbccity.de/schraeglage-bei-einem-motorrad>.
- [18] T. Handson, „BTS7960 Datasheet“, Technology Handson.
- [19] M. Maurer, *Autonomes Fahren Technische, rechtliche und gesellschaftliche Aspekte*, Springer , 2015.

Abbildungsverzeichnis

Abbildung 1: Preisentwicklung von Fahrrädern (2017 - 2021)	9
Abbildung 2: Preisentwicklung von Autos (1980 bis 2021)	9
Abbildung 3: Aufnahme einer Testfahrt des Autonomen Fahrrades (Screenshot)	12
Abbildung 4: Übersicht aller verbauten Komponenten (LTSpice)	15
Abbildung 5: Übersicht der verbauten Komponenten ohne Antriebsmotor (Fritzing)	16
Abbildung 6: BNO055 Lagesensor	17
Abbildung 7: Übersicht über Eulerwinkel	18
Abbildung 8: MD49 Motorsteuerung	18
Abbildung 9: Tabelle für Beschleunigungslevel des EMG49 [14]	19
Abbildung 10: BTS7960 [18]	20
Abbildung 11: Verbauter Levelshifter	20
Abbildung 12: Beaglebone Black	21
Abbildung 13: Veranschaulichung eines 50%igen Tastgrad	23
Abbildung 14: Abbildung der Magneten im Antriebsmotor	24
Abbildung 15: PWM Ausgang des Hinterradantriebs (Ausgabe des Oszilloskop)	24
Abbildung 16: Motorrad ohne Schräglage	26
Abbildung 17: Motorrad in Schräglage	26
Abbildung 18: Beispiel der verschiedenen Schräglagen	27
Abbildung 19: Durchmesser der Teststrecke (Google Earth)	27
Abbildung 20: GoogleEarth aufnahme der Teststrecke (Google Earth)	29
Abbildung 21: Laufzeiten einzelner Komponenten	30
Abbildung 22: Erster Regler	32
Abbildung 23: Pseudocode der ersten Regelung Teil 1	34
Abbildung 24: Pseudocode der ersten Regelung Teil 2	35
Abbildung 25: Zweiter funktionierender Regler	36
Abbildung 26: Pseudocode zur zweiten Regelung	38
Abbildung 27: Testfahrt mit Erster Regelung	41
Abbildung 28: Diagramm bei 10km/h und 1750 als Inputparameter	43
Abbildung 29: Diagramm bei 9km/h und 2000 als Inputparameter	44
Abbildung 30: Diagramm bei 8km/h und 2125 als Inputparameter	45
Abbildung 31: Diagramm bei 7km/h und 2250 als Inputparameter	46
Abbildung 32: Diagramm bei 6km/h und 4500 als Inputparameter	47
Abbildung 33: Nahaufnahme der Schwingung	48
Abbildung 34: Funktion $f_x = 8610x - 3.52 + 400$ für Parameterwahl	48
Abbildung 35: Beaglebone AI	50

Abkürzungsverzeichnis

B

BTS7960 *zweite Motorsteuerung*

E

EMG49 *Lenkmotor*

F

FSD *Full Self Driving*

G

GPIO *General Purpose Input/Output*

I

I2C *Inter-Integrated Circuit*
IMU *BNO055 Lagesensor*

K

KI *Künstlichen Intelligenz*

L

LAN *Local Areal Network*

M

MD49 *Erste Motorsteuerung*
MIT *Massachusetts Institute of Technology*

P

PRU *programmable real-time uni*
PWM *Pulsweitenmodulation*

R

RxD *Receive Data*

S

SCL *Serial clock*
SDA *Serial Data*
SSH *Secure Shell*

T

TxD *Transmit Data*

U

UART *Universal Asynchronous Receiver / Transmitter*

W

WAP *Wifi acces point*
WLAN *Wireless Local Area Network*